



MCT331

Design of Mechatronics 1

Fall 2025

Final Submission

Submitted to

Dr. Shady

Eng. Ahmed Hassabo

Eng. Mina

Submitted by

Ahmed Mohamed Behiry	22P0044
Rewan Mohamed Mostafa	23P0383
Shahd Ahmed Tawfik	23P0169
Ahmed Hisham Mohamed	2200618
Adham Hamada Hazem	22P0100

Submission date

24th of Dec. 2025

Table of Contents

Table of Contents	1
Table of Figures	3
Abstract	5
1 Introduction	6
2 Introduction to CAD design	6
3 Actuation and Motion Subsystem.....	7
3.1 X-Axis: Power Screw Drive	7
3.2 Y-Axis: Belt-Driven Motion.....	7
3.3 Z-Axis: Power Screw and Dual Linear Rails.....	7
4 Cylindrical Product and Reservoir Design	8
5 Product Detection and Classification	8
6 Cartesian Robotic Arm Operation	9
7 Storage Unit Design.....	9
8 Assembly and Integration.....	10
9 Assembly Steps.....	10
10 Materials and Manufacturing processes.....	11
11 Pneumatic Feeding Mechanism	12
12 Control Box Design and Components	12
14 Design Tools and Validation	14
15 Summary	14
16 Model Preparation.....	15
17 Subsystem Definition and Constraints.....	15
17.1 First product base (Ncylinder 1)	16
17.2 Second product base (Ncylinder 2).....	18
17.3 First and Second product lid (Ninnercylinder 1&2)	20
18 Signal-Based Joint Automation	22
19 State flow logic	26
20 Results and Analysis	27
21 Mechanical System	28

21.1	Cartesian Robot Mechanism	28
21.2	Properly Fixed Sensors	28
21.3	Properly Assembled End Effector	29
21.4	Feeding Mechanism	29
21.5	Control Boxes	29
21.6	Mechanical & Safety Considerations.....	29
21.7	Properly Fixed Electronics.....	30
22	Electrical System	30
22.1	Wiring Diagram	30
22.2	Functional On-Off Switch & Emergency Button	31
22.3	Standalone Operation.....	31
22.4	No Breadboard.....	32
22.5	Single Power Source	32
22.6	Cable Management	32
22.7	Wire Labelling	32
23	Stress Analysis on Critical Components.....	33
24	Flow charts.....	49
25	PID Controller Design and Implementation	52
25.1	PID Control Objective	52
25.2	Control Structure.....	52
25.3	Mathematical Model and Feedback	52
25.4	PID Tuning Methodology	52
25.5	Anti-Windup and Dead-Zone Compensation	53
25.6	Real-Time Implementation Using FreeRTOS	53
25.7	Experimental Results and Performance	54
26	Software Implementation – ESP32 (1)	55
26.1	Overview of ESP32 (1) Role	55
26.2	RTOS Task Architecture.....	55
26.3	Inter-Task Synchronization Using Semaphores.....	55
26.4	Product Detection (IR Task)	56
26.5	Feeding Mechanism Control.....	56

26.6	Height-Based Sorting Using Laser Sensors	56
26.7	Color Sorting Using TCS3200 Sensor	57
26.8	ESP-NOW Communication Between Controllers	57
26.9	LCD Feedback and User Interface	57
26.10	System Robustness and Safety	57
27	Software Implementation – ESP32 (1)	58
27.1	System Overview	58
27.2	FIFO Assembly Logic and Part Management	58
27.2.1	Data Structures and System Parameters	58
27.3	Storage Locations and FIFO Queues	59
27.4	Indexing and State Flags	59
27.5	X-Axis: Dual Stepper Motor Control	59
27.6	Y-Axis: Encoder-Based Closed-Loop PID Control	61
27.7	Z-Axis: Vertical Stepper Control	62
27.8	Homing Sequence and Reference Initialization	62
27.9	Absolute Motion Abstraction	63
27.10	End-Effector (Suction) Control	64
27.11	Pick-and-Place Motion Sequences	64
27.12	Assembly Supervisor Logic	65
27.13	RTOS Architecture and Task Allocation	66
27.14	System Initialization and Hardware Configuration	66
27.15	Importance and Contribution to Overall System	67
28	Conclusion	67
29	References	68

Table of Figures

Figure 1	System Assembled	5
Figure 2	CAD system assembly	6
Figure 3	Cartesian Robotic arm cad	7
Figure 4	Laser receiver	8
Figure 5	Laser emitter	8

Figure 6 Color sensor.....	8
Figure 7 Suction cup	9
Figure 8 Storage.....	9
Figure 9 The opposite laser sensor holder	10
Figure 10 Laser Sensor Holder	10
Figure 11 Pneumatic cylinder	12
Figure 12 Feeder	12
Figure 13 Esp32	12
Figure 14 Emergency stop button	12
Figure 15 Direction control valve	13
Figure 16 H-bridge.....	13
Figure 17 Power supply	13
Figure 18 aluminum extrusion profile	13
Figure 19 Z-axis cart	13
Figure 20 Y-axis cart	13
Figure 21 X-axis cart	13
Figure 22 trapezoidal lead screw	14
Figure 23 nut housing	14
Figure 24 Bracket.....	14
Figure 25 Coupling	14
Figure 26 Y end support	14
Figure 27 Pully.....	14
Figure 28 System Simulation Model	15
Figure 29 First product face contact forces.....	16
Figure 30 Cartesian Robot Mechanism.....	28
Figure 31 IR sensor fixation	28
Figure 32 Laser and Color sensor Fixation.....	28
Figure 33 Feeding mechanism.....	29
Figure 34 Wiring diagram.....	30
Figure 35 Emergency button.....	31
Figure 36 On-Off switch.....	31
Figure 37 Wire Labelling.....	32
Figure 38 forward dc.....	54
Figure 39 reverse dc.....	54

Abstract

This project presents the design, development, and implementation of a fully automated Cartesian robotic sorting and assembly system using a Real-Time Operating System (RTOS). The system integrates a pneumatic feeding mechanism, dual laser height detection, an analog color sensor, and a three-degree-of-freedom Cartesian robotic arm equipped with a suction-based end effector. The control architecture relies on activity-based sequencing supported by FreeRTOS tasks and inter-task communication using semaphores to ensure deterministic and predictable behavior. A 5/2 solenoid valve controlled via relay drives the pneumatic actuator, while product classification is achieved through color and height sensing. The system communicates wirelessly with a secondary ESP32 using ESP-NOW to display real-time operational data on an LCD module. A comprehensive mechanical design was developed to be identical to real-life dimensions and assembly constraints, supported by stress analysis to validate structural stability. The final system demonstrated reliable sorting, smooth actuation, accurate classification, and robust real-time communication.

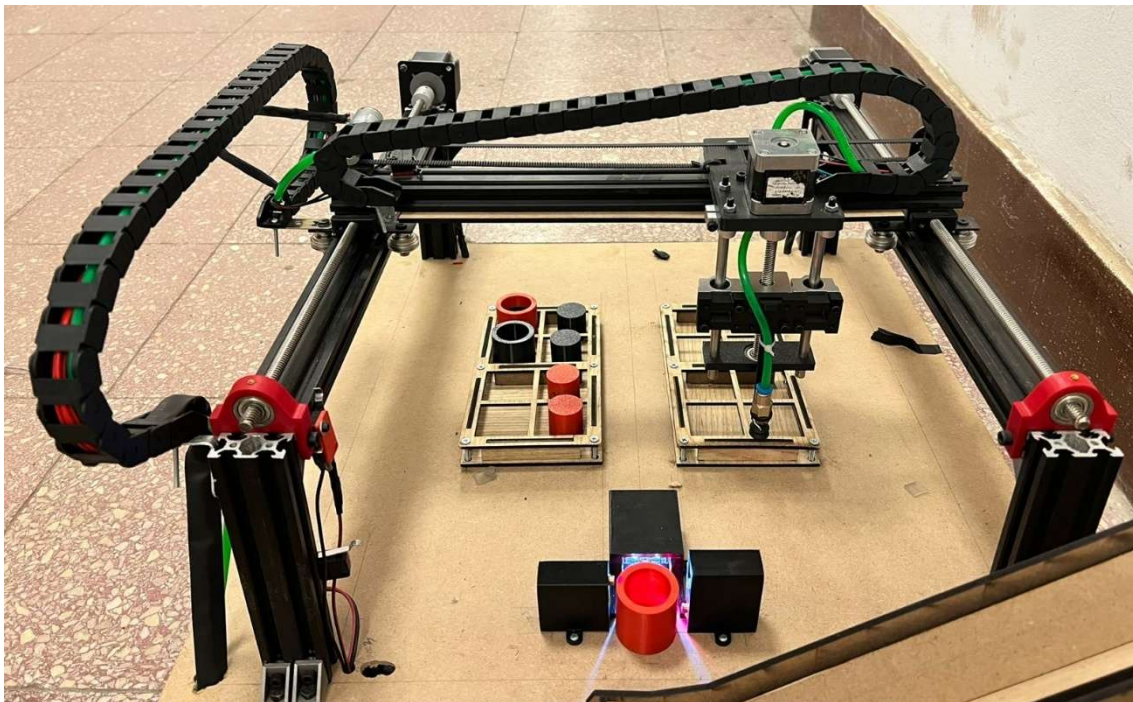


Figure 1 System Assembled

1 Introduction

Automated sorting systems are widely used in industry to improve accuracy, reduce manual labor, and increase production efficiency. This project aims to design and build a compact, fully functional sorting station using a Cartesian robot driven by stepper motors and a pneumatic feeding mechanism. Sensors including IR, dual laser barriers, and a color sensor enable product classification by presence, height, and color. FreeRTOS is used to organize operations into tasks for IR detection, feeding, and sorting, ensuring deterministic, activity-based control. ESP-NOW wireless communication links the main controller to a remote LCD for real-time monitoring. The work combines mechanical, electrical, and embedded design into a complete automated solution.

2 Introduction to CAD design

The CAD design for this project focuses on the complete mechanical layout of an automated classification and storage system. The design integrates multiple subsystems including motion control, sensing, actuation, and product storage into a cohesive robotic platform. Most of the components were derived from and customized to suit the specific functional and dimensional requirements of the project. This approach significantly reduced design time and manufacturing costs while allowing modular assembly and easy modification.

The full system consists of four major subsystems:

- **Control Box**
- **Actuators and Motion Subsystems (X, Y, and Z axes)**
- **Robotic Arm Assembly**
- **Reservoir and Storage Units**

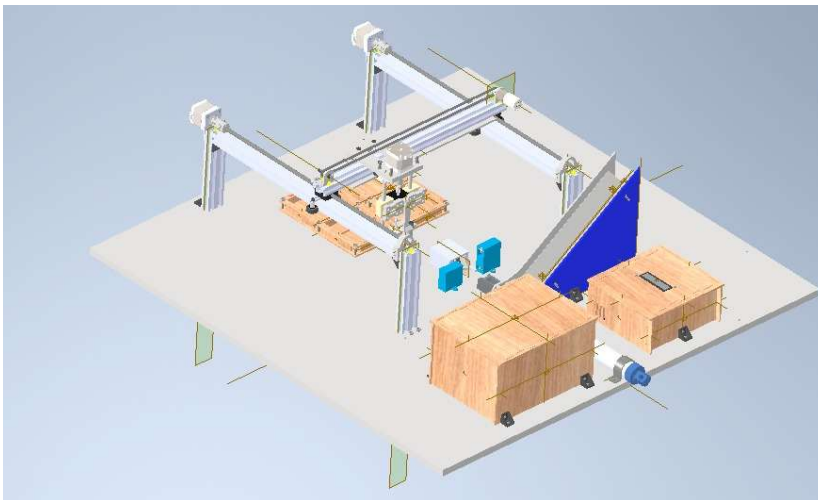


Figure 2 CAD system assembly

3 Actuation and Motion Subsystem

The motion of the robotic arm is achieved using a **Cartesian coordinate mechanism** consisting of three orthogonal linear axes: X, Y, and Z. Each axis was designed to provide high precision, repeatability, and structural rigidity while maintaining compactness.

3.1 X-Axis: Power Screw Drive

- The X-axis is driven by a **power screw** mechanism with an **anti-backlash nut** to ensure smooth motion and eliminate play during directional changes.
- This configuration provides precise horizontal movement and supports accurate positioning for product pickup and placement.

3.2 Y-Axis: Belt-Driven Motion

- The Y-axis is powered by a **DC motor** coupled to a **belt and cart system**, providing lightweight and high-speed motion.
- The belt-driven design minimizes inertia and enables efficient transfer of the Z-axis assembly across the working area.
- This axis serves as the main travel platform for the vertical (Z-axis) mechanism.

3.3 Z-Axis: Power Screw and Dual Linear Rails

- Mounted on the Y-axis cart, the Z-axis provides vertical motion using a **lead screw** system guided by **two linear rails**.
- The dual-rail setup prevents lateral tilting and ensures smooth, stable movement along the vertical direction.
- This design is essential for precise pick-and-place operations performed by the robotic arm.



Figure 3 Cartesian Robotic arm cad

4 Cylindrical Product and Reservoir Design

The products handled by the robotic arm are designed as modular cylindrical components. Each product can be either a base or a lid, and both types are equipped with magnetic interfaces that allow easy stacking and assembly during operation.

A vertical reservoir was developed to serve as the product feeder:

- Cylindrical products are stored in the reservoir in vertical stacks.
- A pusher mechanism releases one product at a time from the reservoir into the classification area.
- The design uses gravity-assisted feeding to simplify the mechanical system and reduce power consumption.

This reservoir serves as the starting point for the automated handling sequence.

5 Product Detection and Classification

Once a product is pushed from the reservoir, it moves into the classification zone equipped with optical and proximity sensors.

- The sensors identify whether the part is a base or lid using two laser sensors that detect height difference and determine its color using a color sensor (among two predefined colors).
- The detection results are transmitted to the control system, which directs the robotic arm to the correct storage position.

This stage ensures that all components are sorted accurately before being transferred, preventing misplacement and ensuring efficient organization of different product types.

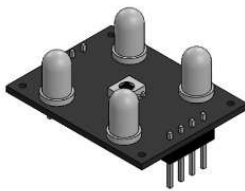


Figure 6 Color sensor

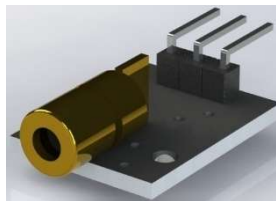


Figure 5 Laser emitter



Figure 4 Laser receiver

6 Cartesian Robotic Arm Operation

After classification, the Cartesian robotic arm is activated to perform precise pick-and-place operations:

- The arm approaches the detected product using coordinated motion across the X, Y, and Z axes.
- Once aligned, the suction cup engages to lift the product.
- The arm then transfers the item to its designated slot in the storage unit based on sensor classification data.

The motion sequence is optimized to minimize travel time while ensuring smooth, collision-free paths. The integration of rigid mechanical guides and accurate actuation allows consistent performance across repeated cycles.

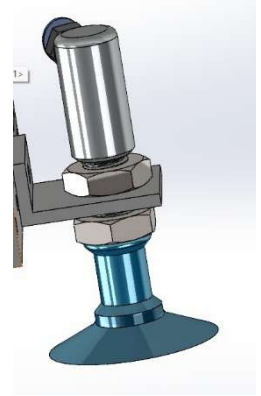


Figure 7 Suction cup

7 Storage Unit Design

The storage system is divided into multiple sections to handle various products and operational states:

- **Main Storage Units:** primary storage module, containing eight dedicated slots, four for each color. are used for organized placement of finished products.
- **Temporary Storage Units:** temporary storage module, also with eight dedicated slots, four for each color., are used as buffer zones in cases where lids arrive before bases or vice versa.

This dual storage concept allows the system to handle asynchronous product arrival, ensuring smooth and uninterrupted operation even when product sequences are irregular. The design supports direct access by the robotic arm for both deposition and retrieval tasks.

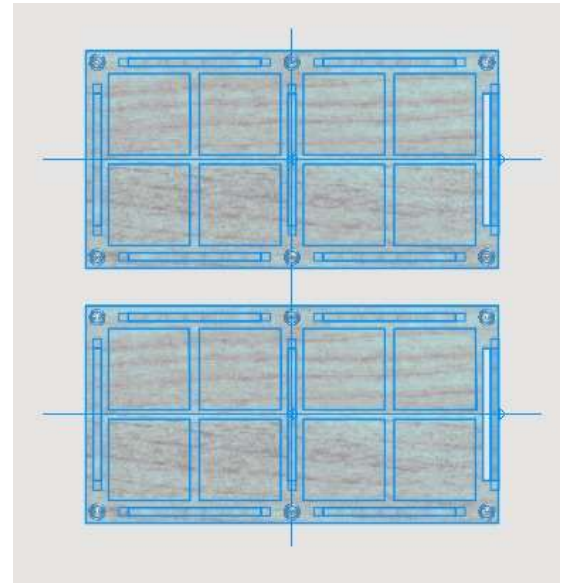


Figure 8 Storage

8 Assembly and Integration

All subsystems including motion axes, reservoir, and storage units are mounted on a rigid base frame designed for structural balance and ease of assembly.

- Standard fasteners were used for modular connections, enabling quick disassembly and reconfiguration.
- Mounting points for sensors and actuators were integrated directly into the CAD design to simplify wiring and assembly.

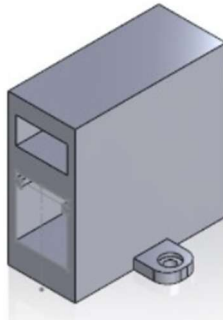


Figure 10 Laser Sensor Holder

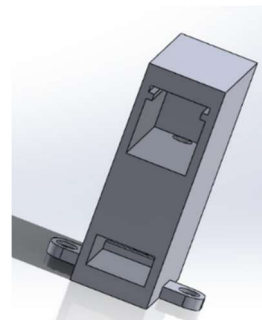


Figure 9 The opposite laser sensor holder

9 Assembly Steps

1. Build the aluminum frame using L-clamps and bolts.
2. Install guide rods and lead screw with bearings.
3. Mount servo motor and connect it to the lead screw via coupler.
4. Assemble and fix the acrylic carts on guide rods.
5. Mount cylinder head, base, and feeder on the cart.
6. Assemble the wooden reservoir and attach to the frame.
7. Fix sensor housings in their positions and install sensors.
8. Mount pulleys and tension the timing belt.
9. Connect all wiring and perform functional test.

10 Materials and Manufacturing processes

No.	Part Name	Material	Process	Qty	Remarks
1	Cylinder Head	PLA / PETG	3D Printed	1	Custom design
2	Cylinder Base	PLA / PETG	3D Printed	1	Custom design
3	Feeder	PLA / PETG	3D Printed	1	Custom design
4	Reservoir (4 Parts)	Plywood 6 mm	Laser Cut	4	Glued assembly
5	Sensor Housings	PLA / PETG	3D Printed	4	For laser, color, IR sensors
6	Aluminum Extrusions	Aluminum	Cut & Drilled	As req.	Frame structure
7	Power Screw	Steel	Cut & Machined	1	Drives cart
8	Lead / Guide Rods	Steel	Cut	2	Support cart motion
9	Carts	Acrylic	Laser Cut	1–2	Holds moving parts
10	Bearings	Steel	Purchased	2	For lead screw ends
11	Coupler	Aluminum	Purchased	1	Connect motor to screw
12	Servo Motors	-	Purchased	1–2	For motion
13	Timing Belt & Pulleys	Rubber / Aluminum	Purchased	Set	Power transmission
14	Screws, Nuts, Washers	Steel	Purchased	As req.	Assembly fasteners
15	L-Clamps & C-Clamps	Steel	Purchased	As req.	Fix parts to frame
16	Wires & Connectors	Copper	Purchased	As req.	Electrical wiring

11 Pneumatic Feeding Mechanism

The product feeding system is driven by a pneumatic cylinder responsible for transferring cylindrical components (bases or lids) from the vertical reservoir into the classification area through the feeder.

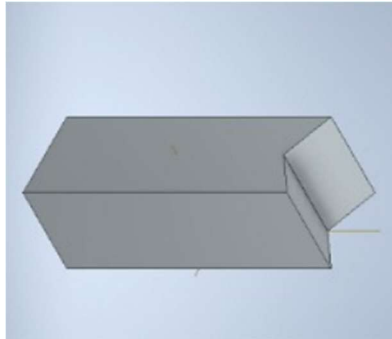


Figure 12 Feeder

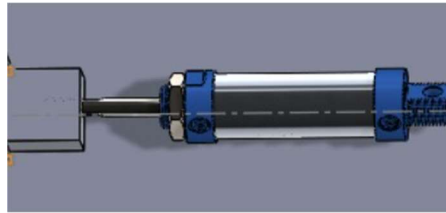


Figure 11 Pneumatic cylinder

12 Control Box Design and Components

The **control box** serves as the main electrical and logical interface of the system. It houses all key electronic components required for motion control, pneumatic actuation, and safety management.

Main Components

1. Dual ESP32 Microcontrollers:

- One ESP32 is dedicated to handling sensor data acquisition and product classification, while the second controls the motion and actuation commands for the robotic arm and pneumatic cylinder.
- The two boards communicate via serial or I²C interface to ensure real-time coordination.



Figure 13 Esp32

2. Emergency Stop Button:

- A manually operated emergency stop switch instantly cuts off power to actuators and pneumatic valves in case of abnormal operation.
- The circuit is designed as a normally closed (NC) loop to ensure fail-safe behavior.



Figure 14Emergency stop button

3. Direction Control Valve:

- A 5/2 solenoid-operated valve controls the pneumatic cylinder movement, directing airflow for extension or retraction.
- The valve is driven by the ESP32

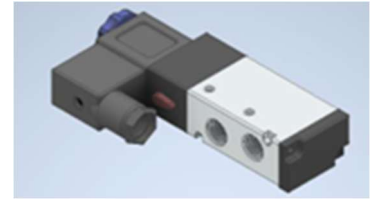


Figure 15 Direction control valve

4. H-Bridge Motor Driver:

- Controls the DC motor on the Y-axis (belt-driven system) by managing direction and speed through pulse-width modulation (PWM).
- The H-Bridge allows smooth acceleration and deceleration to protect mechanical parts and ensure precise motion.



Figure 16 H-bridge

5. Power Supply Unit:

- Provides stable 12V regulated outputs for the ESP32 boards, sensors, solenoid valves, and motor drivers.
- An internal overcurrent and thermal protection circuit safeguards all components from overload.



Figure 17 Power supply

13 3.4 Robotic Arm Structural Design

The **robotic arm structure** is constructed from **20×45 mm aluminum extrusion profiles**, chosen for their lightweight, high strength, and modular assembly properties.



Figure 18 aluminum extrusion profile

Frame and Mounting

- The aluminum sections form the primary support for the X, Y, and Z motion axes.
- **Corner brackets and joining plates** secure the extrusions, ensuring rigid alignment and vibration resistance.
- The modular nature of the profiles allows easy reconfiguration or extension of the robotic arm structure in future iterations.



Figure 21 X-axis cart

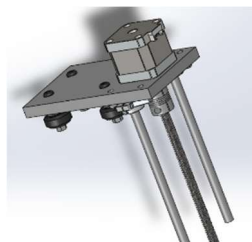


Figure 20 Y-axis cart

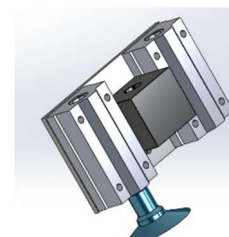


Figure 19 Z-axis cart

Axis Assembly

- Motion is driven by a trapezoidal lead screw paired with a lead screw nut and nut housing to convert rotary motion into precise linear displacement.
- Flexible shaft couplings connect the motor output shaft to the lead screw, compensating for minor misalignments and reducing mechanical stress.
- The assembly is mounted on aluminum extrusion rails using custom-designed brackets that maintain correct alignment and load distribution.
- End supports are placed on both ends of the lead screw to absorb axial loads and maintain rigidity during operation.

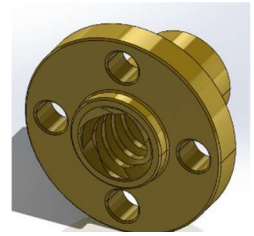


Figure 22 trapezoidal lead screw

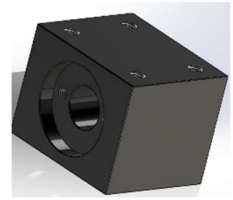


Figure 23 nut housing

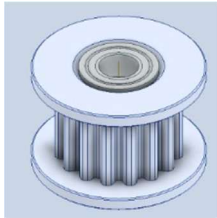


Figure 27 Pulley

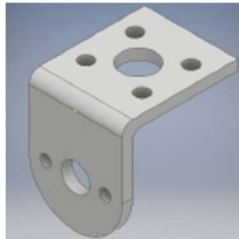


Figure 24 Bracket

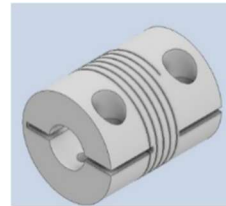


Figure 25 Coupling

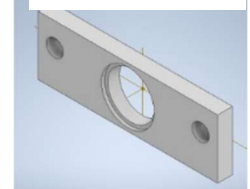


Figure 26 Y end support

14 Design Tools and Validation

The 3D modeling was performed in **SolidWorks**. Each subsystem was modeled as an independent assembly and later integrated into the complete robotic system.

- All parts were exported as **STEP (.stp)** files for cross-platform compatibility.
- **Exploded views** and **sectional analyses** were created to visualize internal assemblies and confirm clearances between moving parts.

15 Summary

The CAD design successfully integrates all essential components of the automated classification and storage system. By combining open-source parts, custom-designed cylindrical products, magnetic assemblies, and a Cartesian robotic arm, the design achieves a balance between functionality, precision, and modularity.

The vertical reservoir enables automatic feeding, the sensor array ensures accurate classification by type and color, and the robotic arm efficiently organizes products into permanent or temporary storage units. This design serves as the foundation for the control, sensing, and automation systems developed in subsequent project stages.

16 Model Preparation

The process began with obtaining the full CAD assembly of the designed system. The CAD files were prepared using SolidWorks and exported in a compatible format for MATLAB Simscape Multibody. Each of the four main subsystems was imported as a separate rigid body to enable independent motion and interaction control.

Before running the simulation, the mechanical constraints for each subsystem were defined based on their physical function. Joints, links, and contact surfaces were carefully configured to represent the actual assembly behavior, ensuring that simulated movements accurately mirrored real-world conditions.

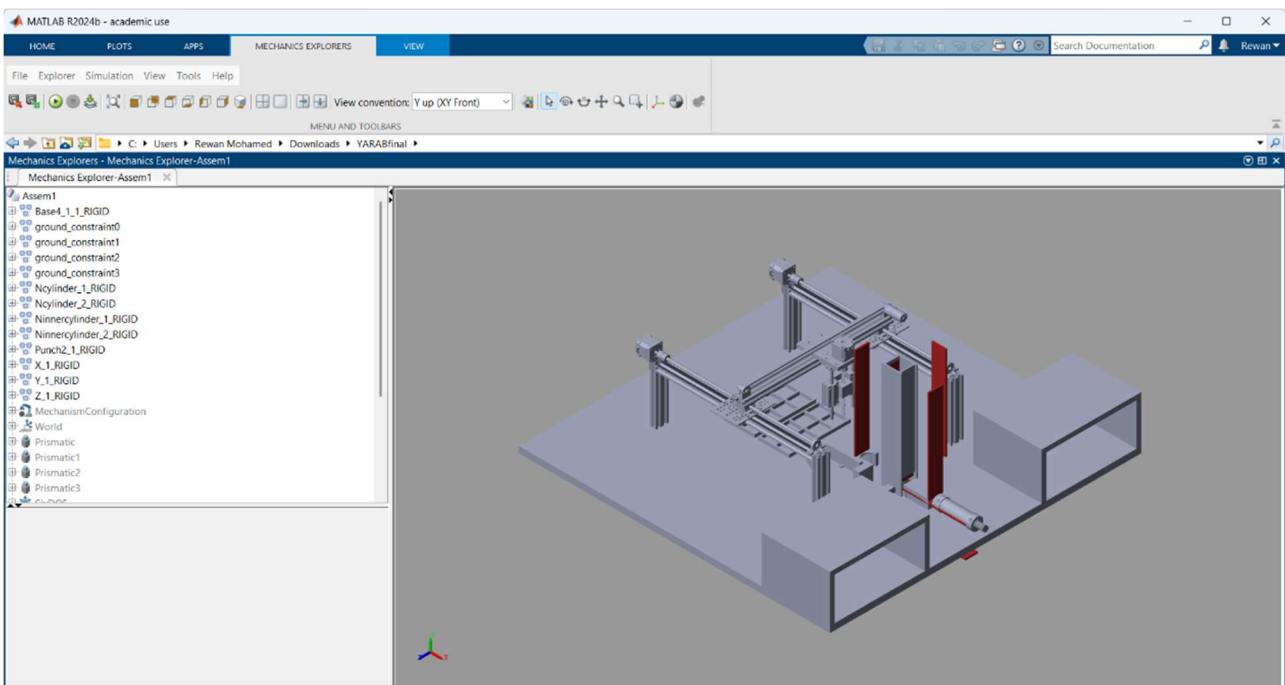


Figure 28 System Simulation Model

17 Subsystem Definition and Constraints

This section defines the mechanical subsystems and their constraints used to simulate the interaction between the main components, including the base, piston, cylinders, reservoir, and suction cup. The constraints are implemented to prevent interpenetration, ensure correct motion limits, and model realistic contact and suction forces.

Rigid transform blocks are applied to align the coordinate systems of each interacting part, while bus selectors and delay elements are used to extract and synchronize the normal force signals during simulation. This configuration provides an accurate representation of the system's physical behavior and the interaction between both products under various loading conditions.

17.1 First product base (Ncylinder 1)

Figure X illustrates the contact force modeling applied to the first product's base within the simulation. The model defines multiple contact constraints to prevent penetration between interacting bodies and to simulate realistic physical behavior of the system components.

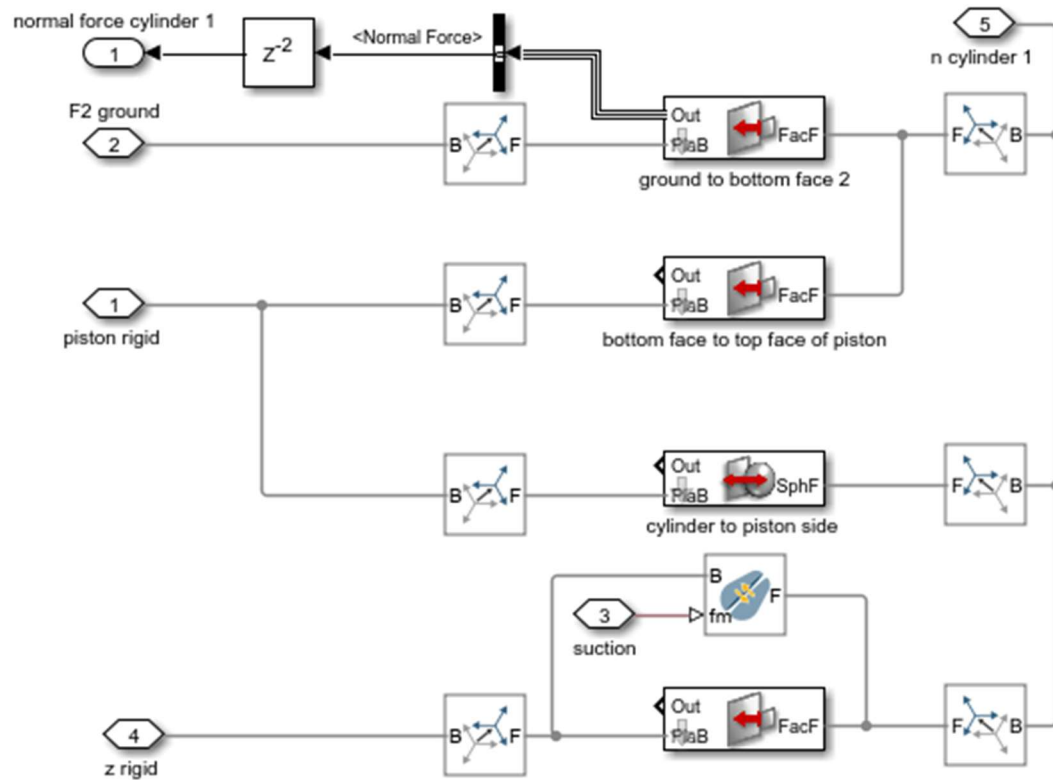


Figure 29 First product face contact forces

Ground Contact Constraint

The first constraint represents the contact between the product base and the ground. Its primary function is to prevent penetration into the ground surface during motion or applied loading. This constraint outputs the normal contact force generated at the point of interaction.

Piston Face Constraint

The second constraint is applied between the bottom face and the top face of the piston. It prevents any interpenetration between these two components during operation, ensuring correct mechanical interaction and maintaining the piston's motion limits.

Piston Side Constraint

The third constraint models the contact between the piston side and the cylinder wall. This interaction restricts lateral penetration and guarantees proper guidance of the piston inside the cylinder. The constraint outputs the corresponding normal and frictional forces acting on the piston side.

Suction Cup Constraint

The final constraint is defined between the product base and the suction cup along the z-axis. An internal force connection is implemented to simulate the suction effect generated during contact. This allows accurate representation of the adhesion force within the system.

Rigid Transforms and Coordinate Alignment

Each of the previously mentioned constraints includes a rigid transform block. These blocks are used to manipulate and align the local coordinate frames of the interacting components to their correct physical positions in the assembly.

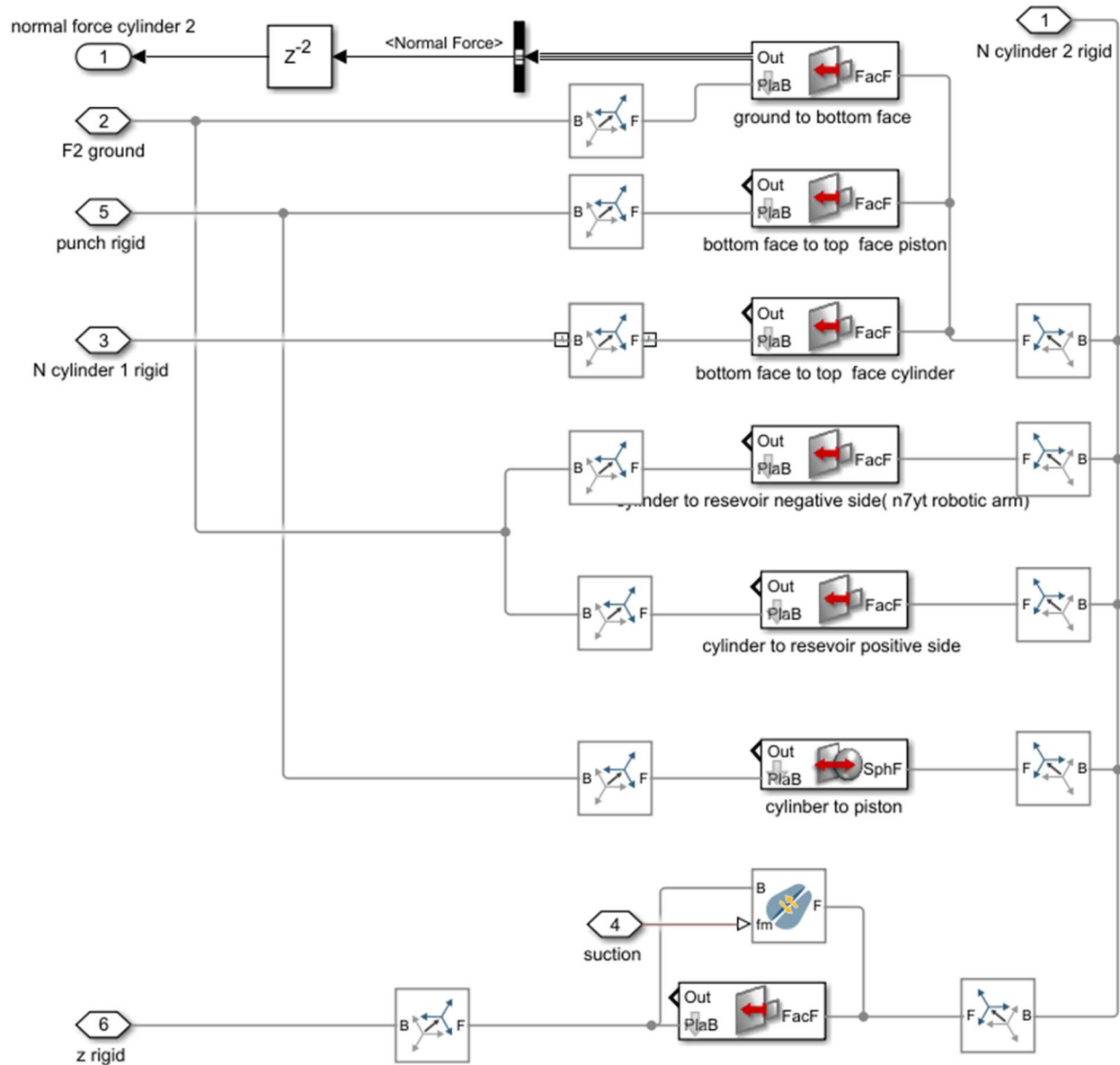
Force Selection and Signal Delay

At the upper section of the model, a bus selector is used to extract one of the four contact forces typically the normal force corresponding to ground contact.

Additionally, a delay block is inserted to account for the initialization phase of the simulation, as the contact force signal does not exist at time zero but appears after the contact is established.

17.2 Second product base (Ncylinder 2)

The subsystem shown in Figure X models contact interactions for the second product's base and its connected components (piston, cylinders, reservoir, and suction cup). The aim is to enforce realistic mechanical constraints, prevent interpenetration between bodies, and simulate contact and suction forces during operation.



Ground and Bottom Face Constraint

A contact constraint is defined between the ground and the bottom face of the base. Its role is to prevent penetration of the base into the ground and to produce the normal reaction force used for support analysis.

Bottom Face to Piston Constraint

A constraint is implemented between the base bottom face and the top face of the piston. This prevents the piston and base from intersecting during motion, ensures proper force transfer, and maintains alignment of axial motion.

Bottom Face to Top Face of Cylinder Constraint

A dedicated constraint is placed between the base bottom face and the top face of the cylinder. This constraint prevents any penetration between the second product's base and the cylinder top face and explicitly enforces that the base does not intrude into the first product's face.

Cylinder–Reservoir Side Constraints

Two contact constraints connect the cylinder to the reservoir negative and positive sides (representing lateral limits and guiding features, for example those associated with the robotic arm).

These constraints restrict lateral displacement of the cylinder, preventing excessive travel and maintaining guided motion within the reservoir bounds.

Cylinder–Piston Side (Spherical) Constraint

A spherical contact constraint models the cylinder-to-piston side interaction. This captures lateral contact behavior and allows computation of reaction forces when side contact occurs, improving fidelity of piston guidance simulation.

Suction Cup (Z-Axis) Constraint and Internal Force

A contact constraint between the base and the suction cup along the z-axis models attachment behavior.

An internal force connection (suction model) is included to represent the adhesion force generated by the cup, enabling simulation of pick-and-place or gripping events.

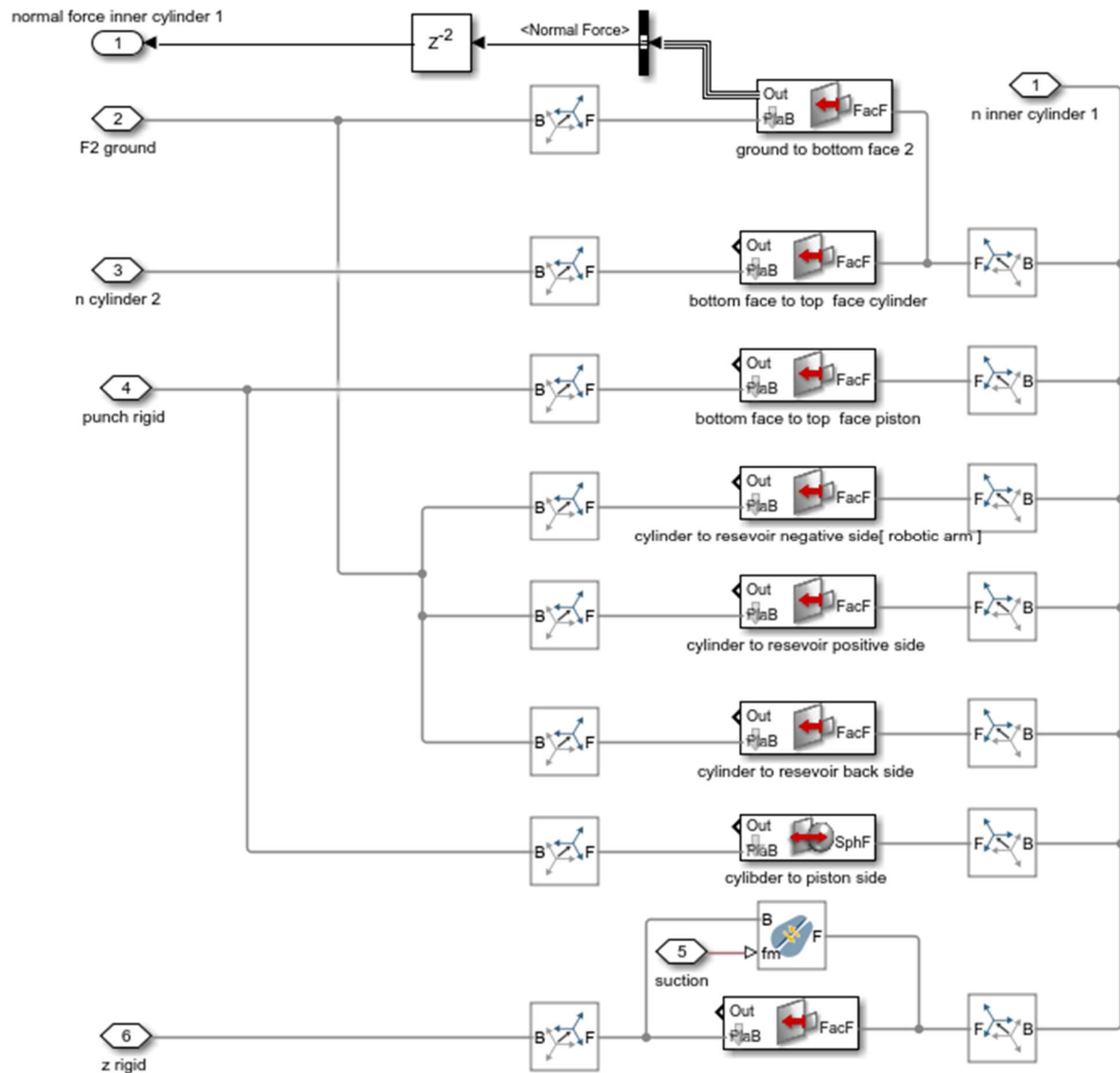
Rigid Transforms, Force Bus and Signal Timing

Each contact pair uses a rigid transform block to correctly align local coordinate frames and position the constraint points.

A bus selector collects multiple contact outputs so that a single contact (e.g., the relevant normal force) can be chosen for monitoring or control. A delay block compensates for the initialization period during which contact forces are not yet present at simulation start.

17.3 First and Second product lid (Ninnercylinder 1&2)

The first product's base model establishes the fundamental mechanical interactions between the system's primary components. Contact constraints are employed to accurately represent physical boundaries, prevent geometric penetration, and enable realistic force transmission within the virtual prototype. The implemented constraints are summarized as follows:



Ground–Bottom Face Constraint

A contact constraint is defined between the ground and the bottom face of the base. Its primary role is to prevent penetration of the base into the ground and to generate the normal reaction force required for structural support and stability analysis.

Bottom Face–Piston Constraint

A constraint is implemented between the base's bottom face and the piston's top face. This constraint ensures proper alignment of the piston's axial motion, prevents interpenetration between components, and facilitates accurate force transfer during actuation.

Bottom Face–Top Face of Cylinder Constraint

A dedicated constraint links the base's bottom face to the top face of the cylinder. It enforces non-penetration between the second product's base and the cylinder's top surface, maintaining clear separation and mechanical integrity between the two structural layers.

Cylinder–Reservoir Side Constraints

Two lateral contact constraints connect the cylinder to the reservoir's negative and positive sides, representing the physical guiding features of the robotic arm mechanism. These constraints restrict lateral movement of the cylinder, ensuring guided motion and preventing excessive displacement beyond the reservoir's boundaries.

Cylinder–Piston Side (Spherical) Constraint

A spherical contact constraint is used to model the cylinder–piston side interaction. This allows lateral contact detection and computation of reaction forces during side loading, improving the fidelity of piston guidance and alignment under off-axis forces.

Suction Cup (Z-Axis) Constraint and Internal Force

A contact constraint along the z-axis is established between the base and the suction cup to simulate attachment and detachment behavior. An internal force element represents the suction or adhesion effect generated by the cup, enabling simulation of gripping or pick-and-place events with realistic force modeling.

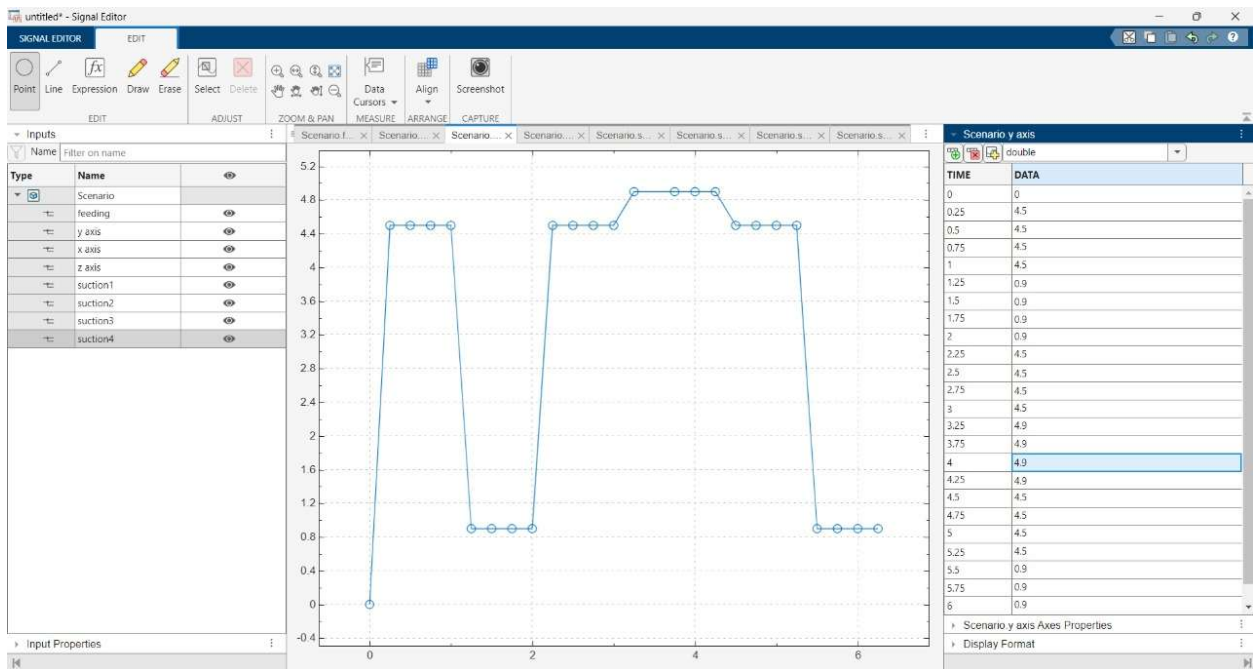
Rigid Transforms, Force Bus, and Signal Timing

Each contact pair employs a rigid transform block to correctly align the local coordinate frames and position constraint points within the assembly. A bus selector is used to aggregate multiple contact outputs, allowing selection of key quantities such as the normal force for monitoring and control purposes. Additionally, a delay block is implemented to compensate for initialization effects, ensuring accurate contact force calculation after simulation start-up.

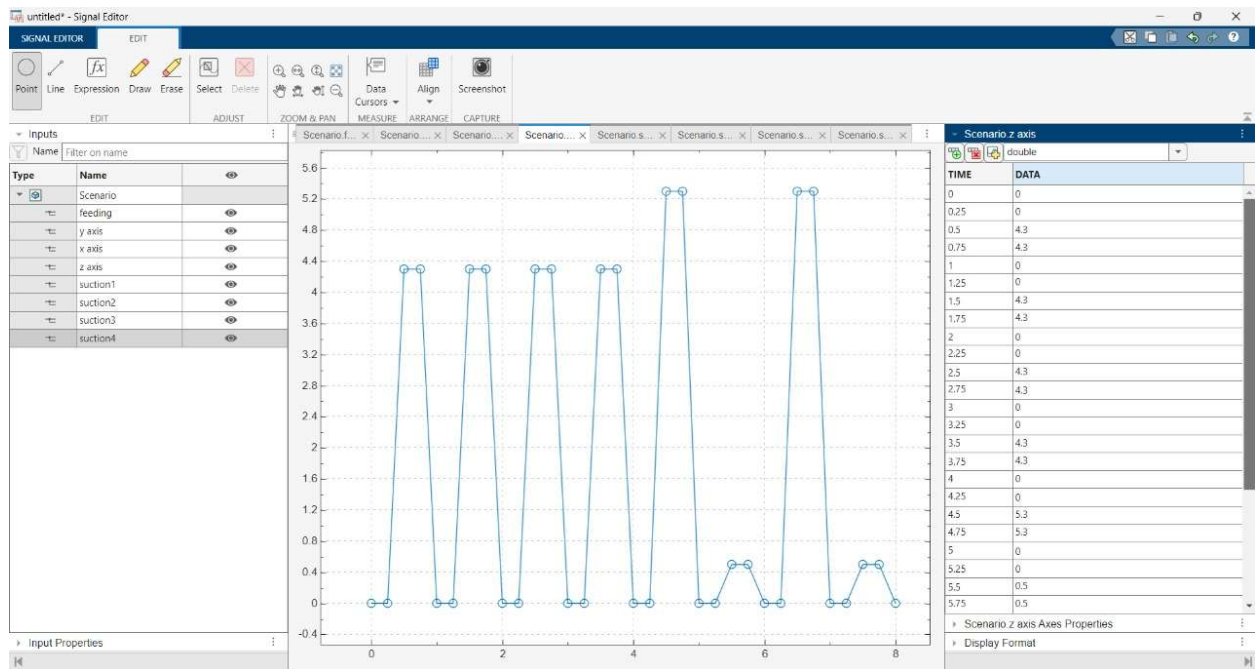
18 Signal-Based Joint Automation



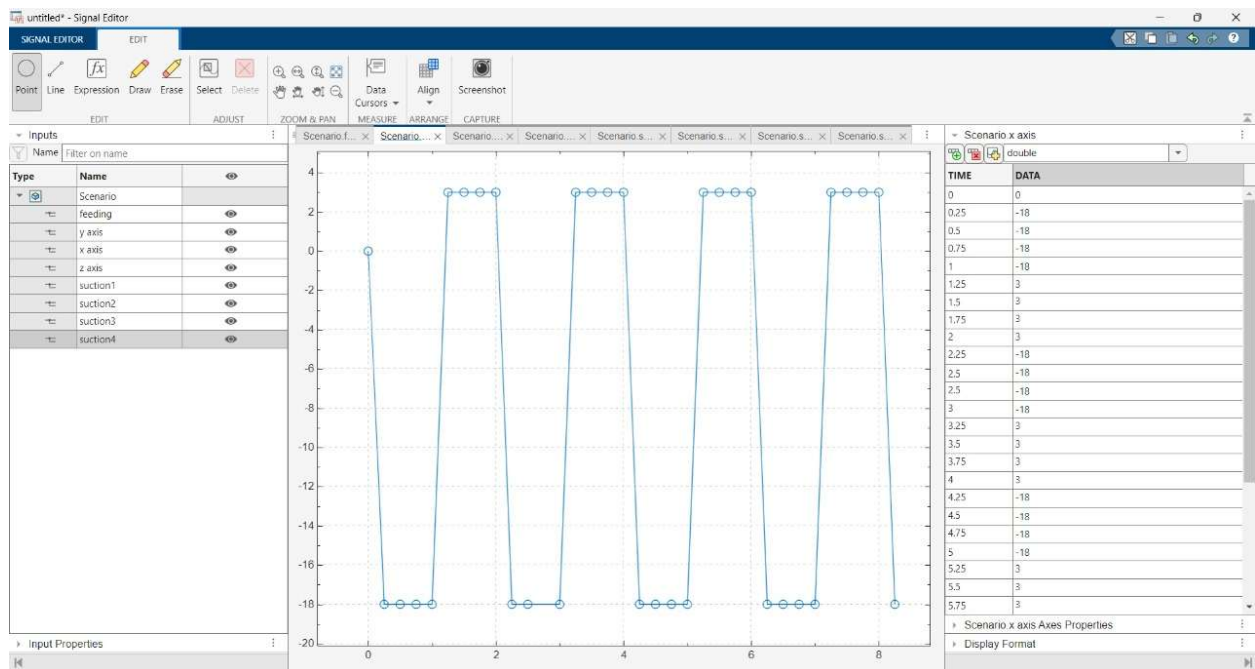
Y-axis



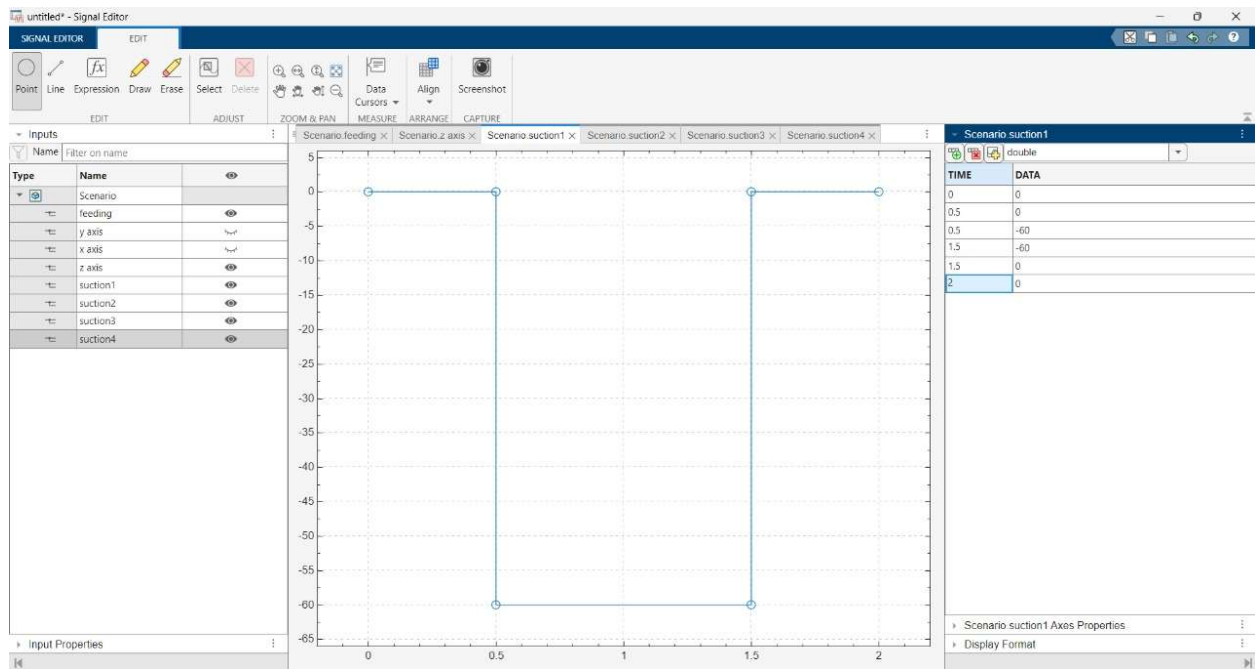
z-axis



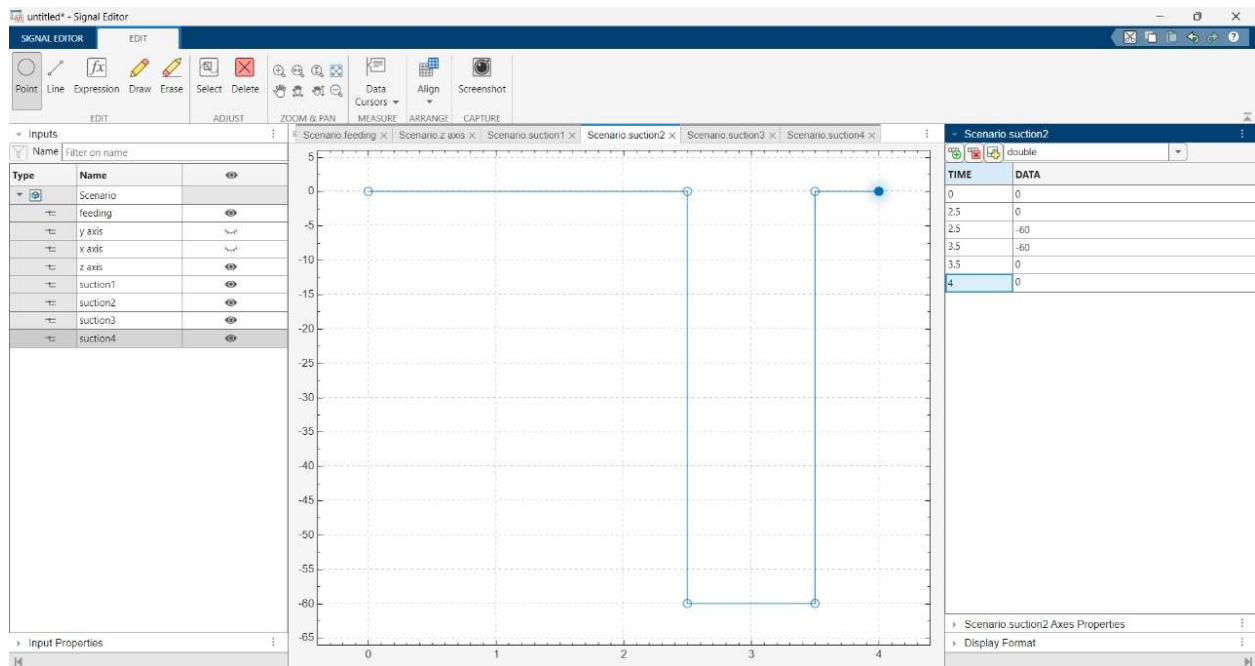
x-axis



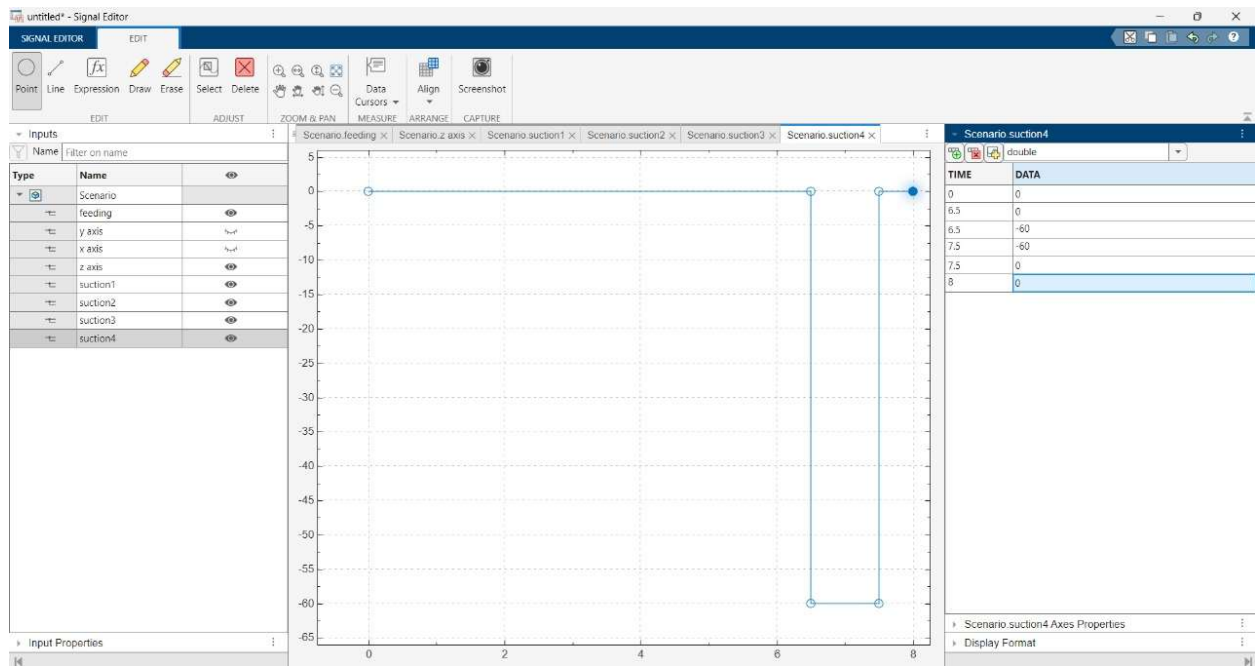
Suction 1



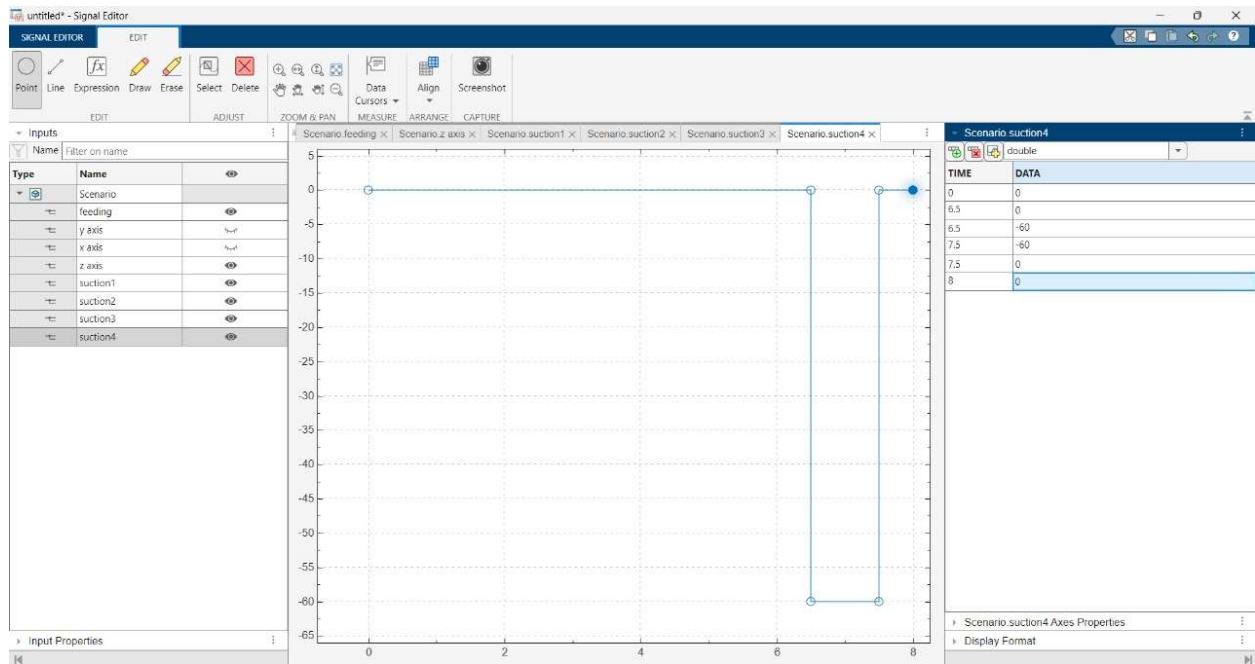
Suction 2



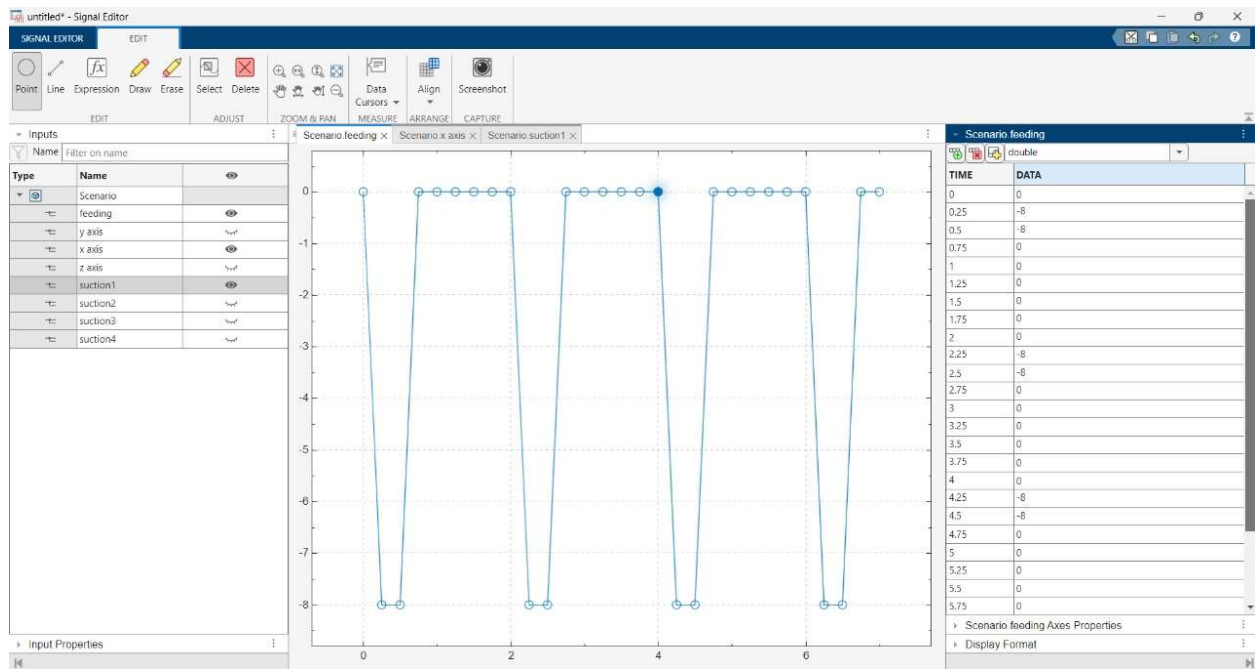
Suction 3



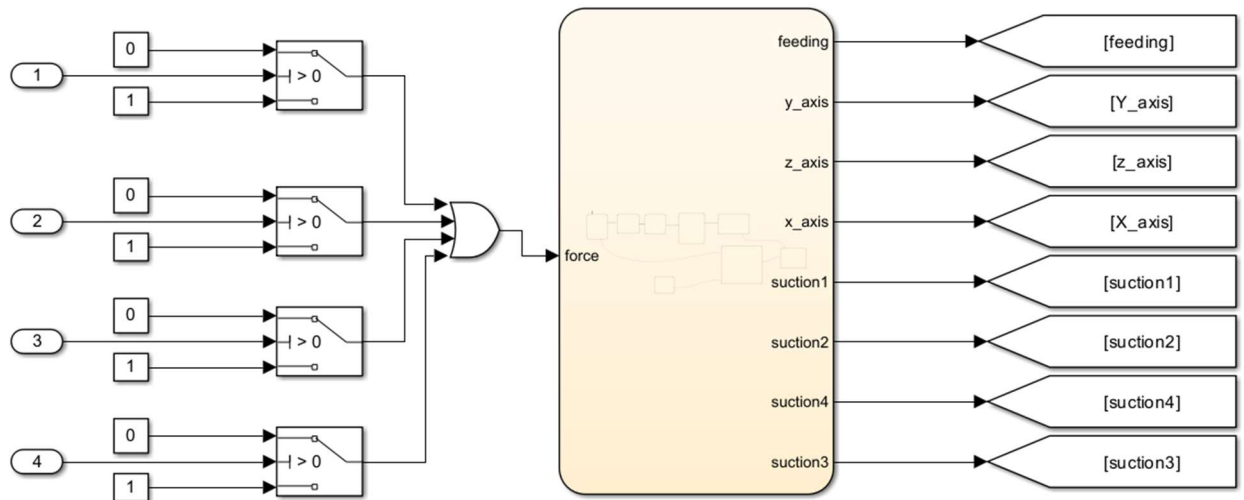
Suction 4

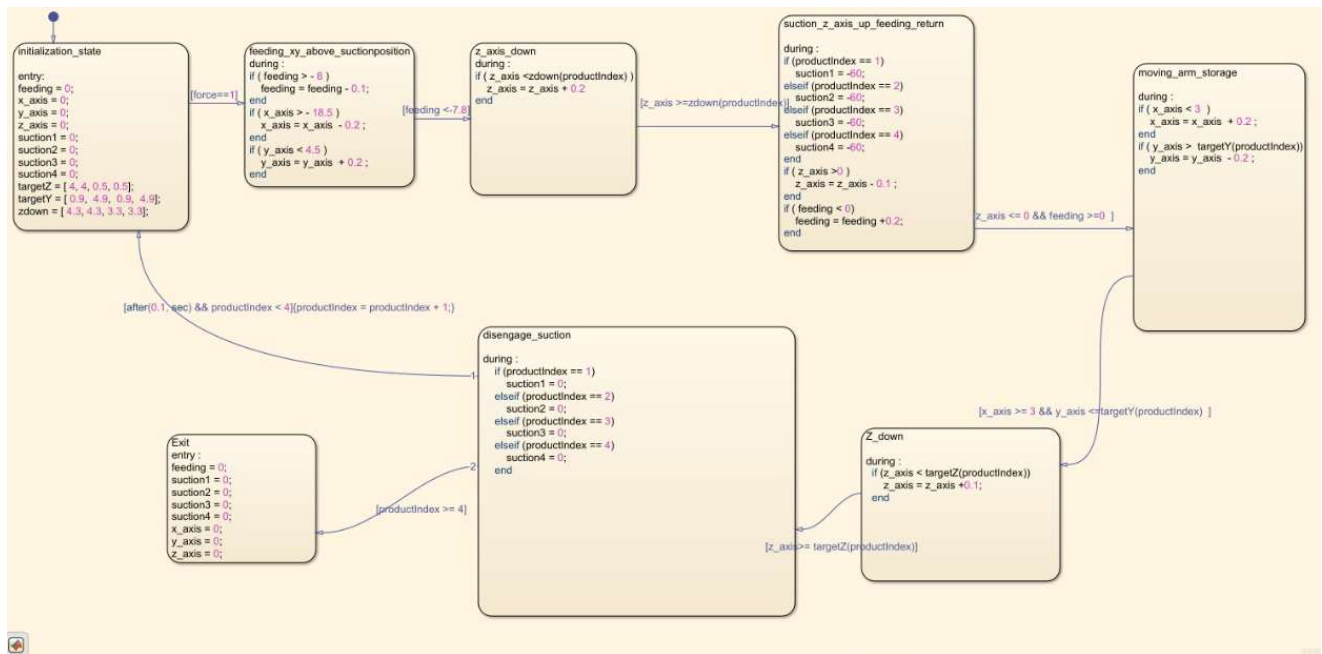


Feeding



19 State flow logic





20 Results and Analysis

The simulation successfully demonstrated mechanical and logical coordination among the four subsystems. The generated contact forces showed realistic responses during engagement, while the control signals achieved precise synchronization across all joints. The State flow loop confirmed the system's capability to operate cyclically, where each cycle produced one complete product and updated the storage array accordingly. This provided strong evidence that the design is functionally sound and ready for physical prototyping.

21 Mechanical System

21.1 Cartesian Robot Mechanism

The Cartesian robot consists of **two stepper-driven linear axes (X and Z)** and a **DC motor Y-axis**, allowing precise point-to-point movement. All rails, couplers, and motion components were aligned and rigidly mounted to ensure smooth, repeatable, and backlash-free motion.

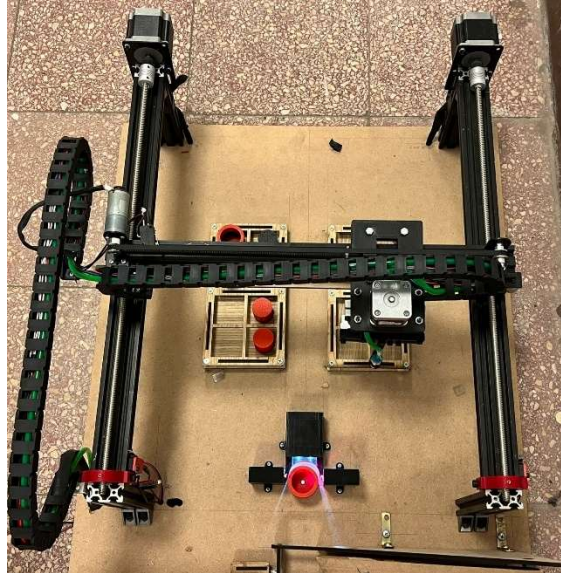


Figure 30 Cartesian Robot Mechanism

21.2 Properly Fixed Sensors

All sensors (IR presence sensor, upper/lower laser receivers, limit switches) were securely mounted in fixed positions with 3D-printed holders to ensure **stable readings**, correct alignment, and vibration resistance.



Figure 32 Laser and Color sensor Fixation

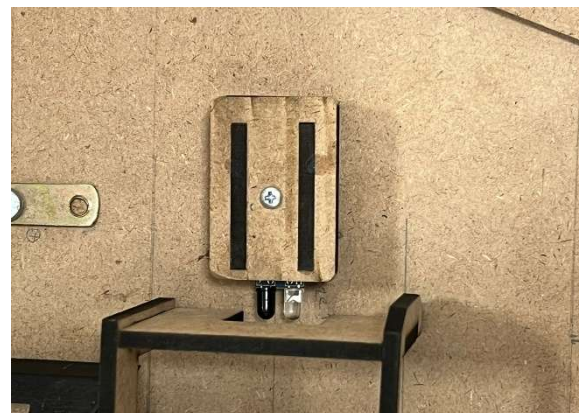


Figure 31 IR sensor fixation

21.3 Properly Assembled End Effector

The suction-cup end effector was mounted on a rigid bracket, with pneumatic tubing neatly routed. The vacuum source and nozzle were aligned to ensure consistent gripping and safe vertical lifting.

21.4 Feeding Mechanism

The feeder uses a **single 5/2 solenoid valve** controlling a pneumatic cylinder. The mechanism extends to push one product forward until it blocks the lower laser, then retracts automatically. The system ensures consistent, single item feeding. The magazine was mounted to keep products stacked inclinedly. Proper fixation ensures smooth feeding and prevents jamming.



Figure 33 Feeding mechanism

21.5 Control Boxes

The control box houses the ESP32, relays, power supply, and wiring. It is designed with a hinge/cover that can be opened for inspection while keeping all components protected and organized during operation.

21.6 Mechanical & Safety Considerations

All parts were deburred and rounded, no sharp edges. No glued parts were used—everything is bolted or screwed. Moving areas are protected, and wiring is kept away from motion components to ensure operator safety.

21.7 Properly Fixed Electronics

All electronic modules (ESP32, relay, drivers, distribution board) are mounted on standoffs inside the control box. Cables are tied, labeled, and strain-relieved to avoid loose connections or accidental disconnecting.

22 Electrical System

22.1 Wiring Diagram

A complete and clearly structured wiring diagram was created showing all connections between sensors, actuators, the ESP32 controller, power supply, relays, and emergency circuits. The diagram follows standard electrical drafting rules, ensuring readability and easy troubleshooting.

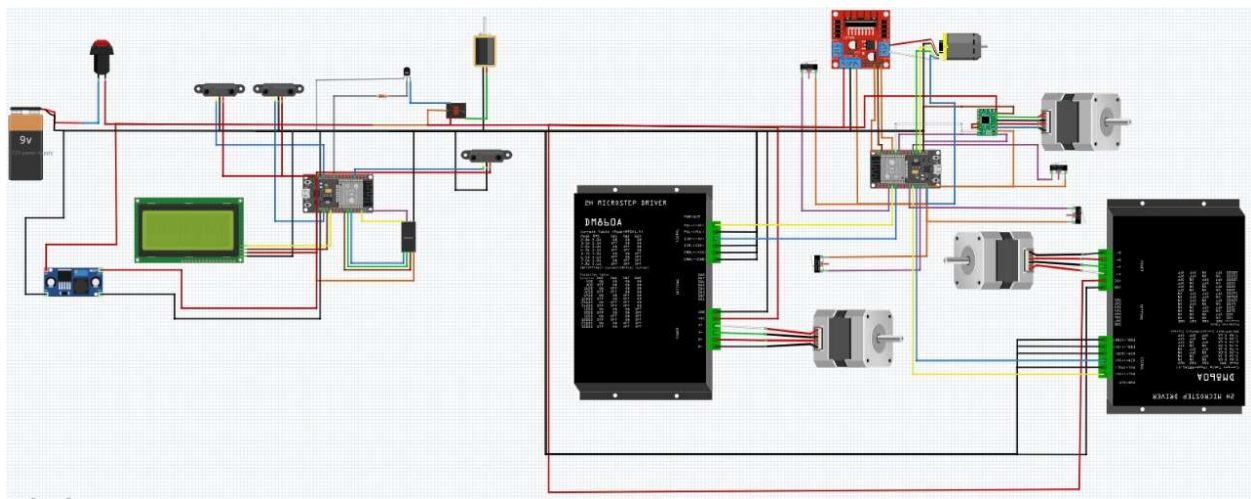


Figure 34 Wiring diagram

Note: The power supply is 12V

22.2 Functional On-Off Switch & Emergency Button

A dedicated emergency stop (E-STOP) button and on-off switch are integrated in series with the power line feeding the actuator system. Pressing the button immediately cuts power to the solenoid valve and halts all robot motion, ensuring safety during operation and maintenance.

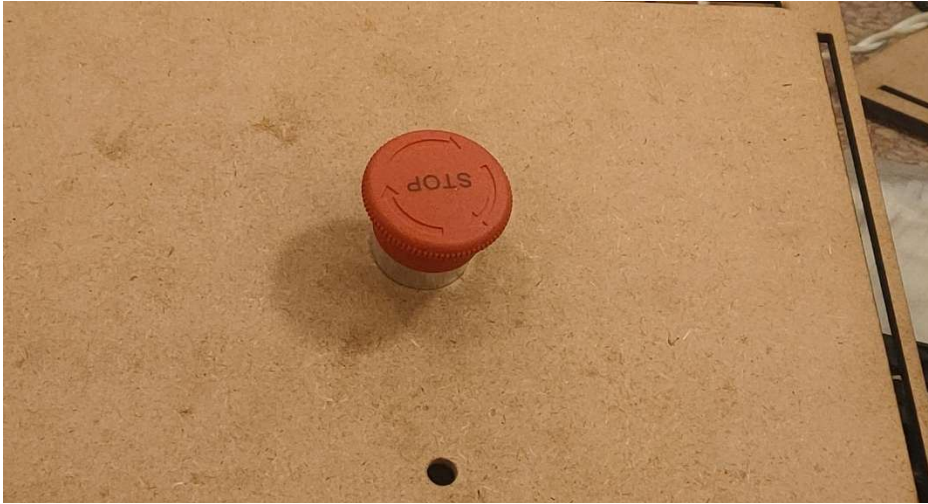


Figure 35 Emergency button



Figure 36 On-Off switch

22.3 Standalone Operation

The electrical system is fully standalone, containing its own power distribution, control electronics, and safety circuits. The ESP32 microcontroller independently executes all RTOS-based tasks without requiring an external computer.

22.4 No Breadboard

All components are hard-wired using soldered terminals and secured connectors. PCBs and terminal blocks are used instead of breadboards to ensure electrical reliability, reduce noise, and withstand vibration.

22.5 Single Power Source

A unified 12V DC power supply is used to power all components, including sensors, relay modules, and the ESP32. This ensures stable operation, simplifies wiring, and minimizes ground-loop issues.

22.6 Cable Management

Cables are routed neatly using cable trays, sleeves, and zip ties. Power and signal wires are separated to reduce electromagnetic interference. All wiring is organized to prevent tangling, enhance airflow, and maintain accessibility.

22.7 Wire Labelling

Every wire is labeled using heat-shrink or printed tags indicating its function (e.g., “IR”, “Laser_Lower”, “Valve_Relay”, “5V”, “GND”). Proper labeling simplifies debugging, maintenance, and future upgrades.

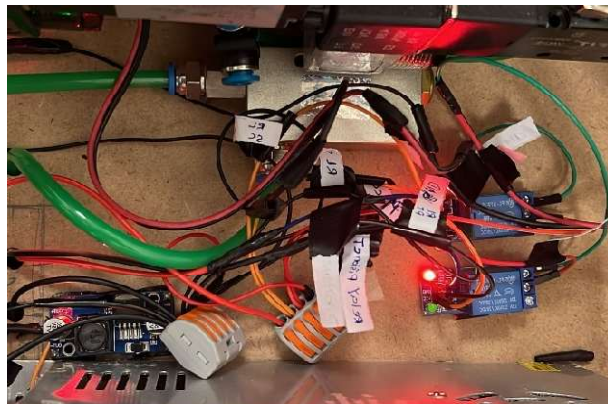
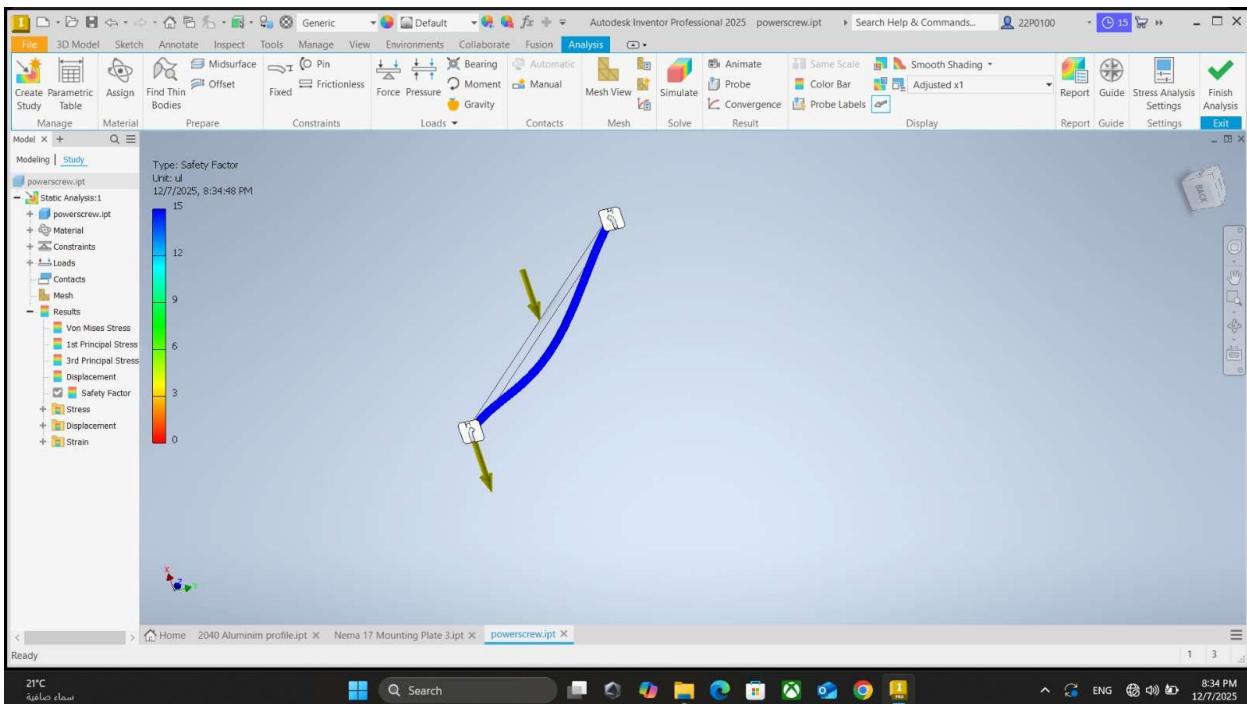
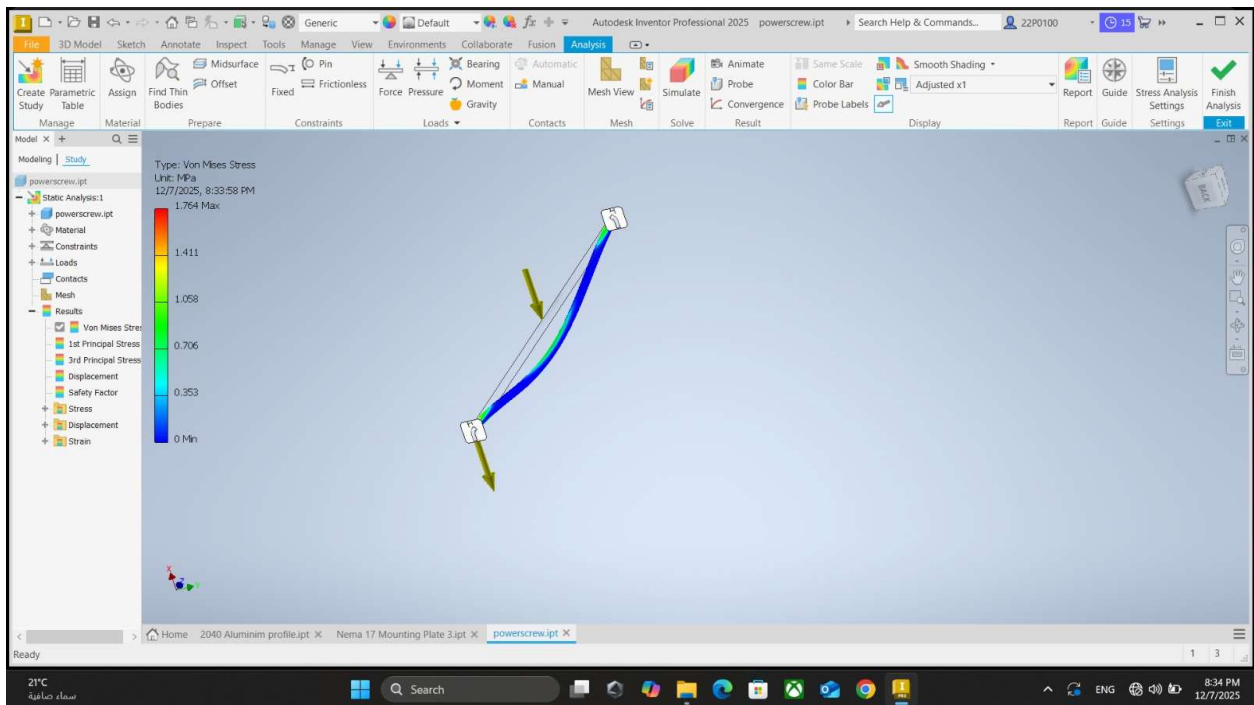
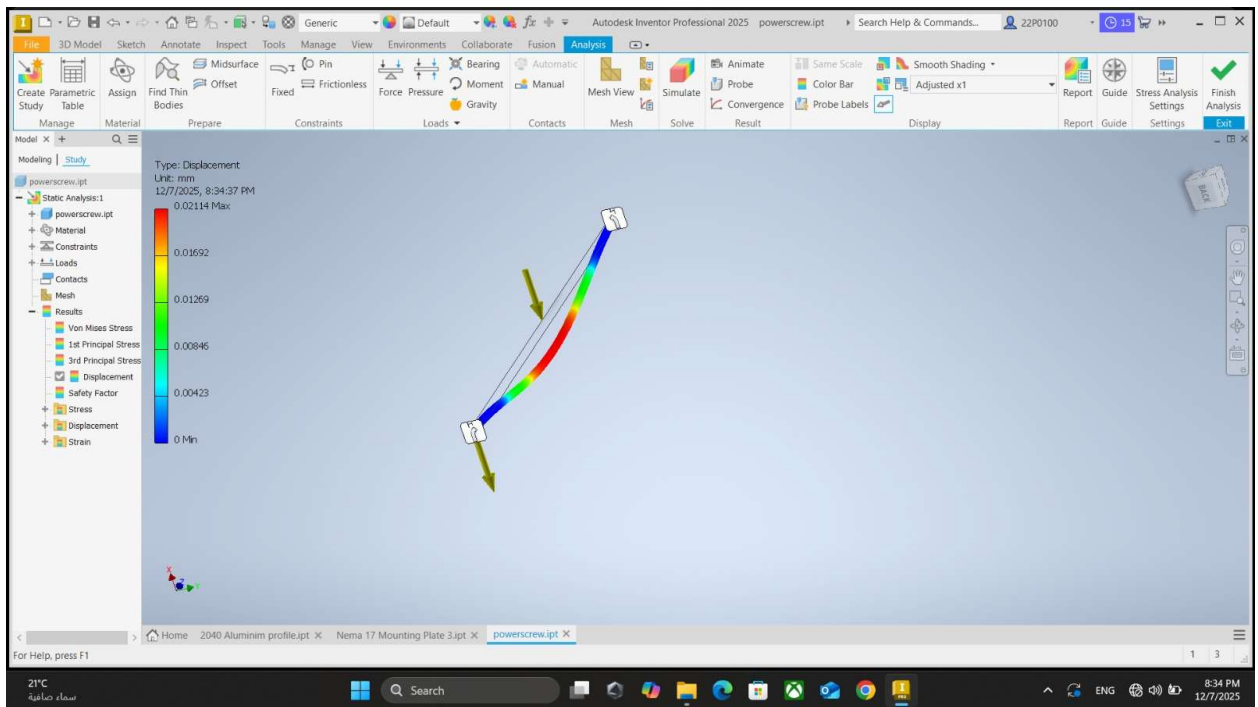


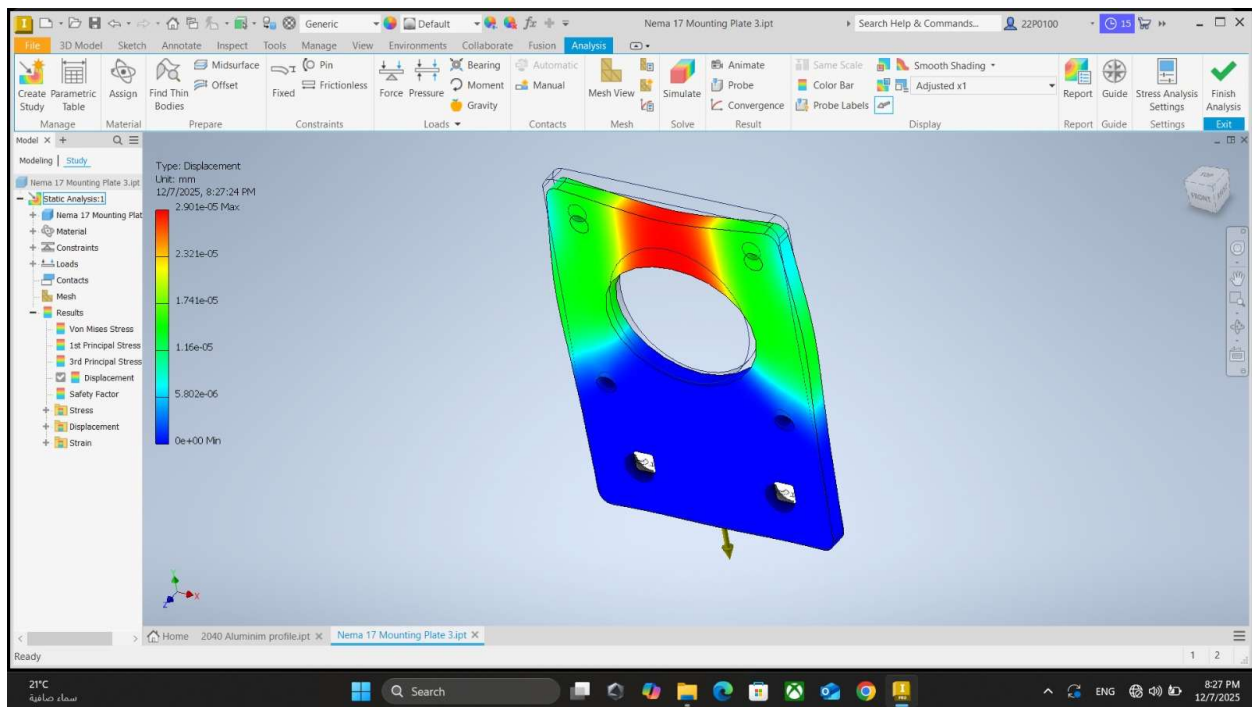
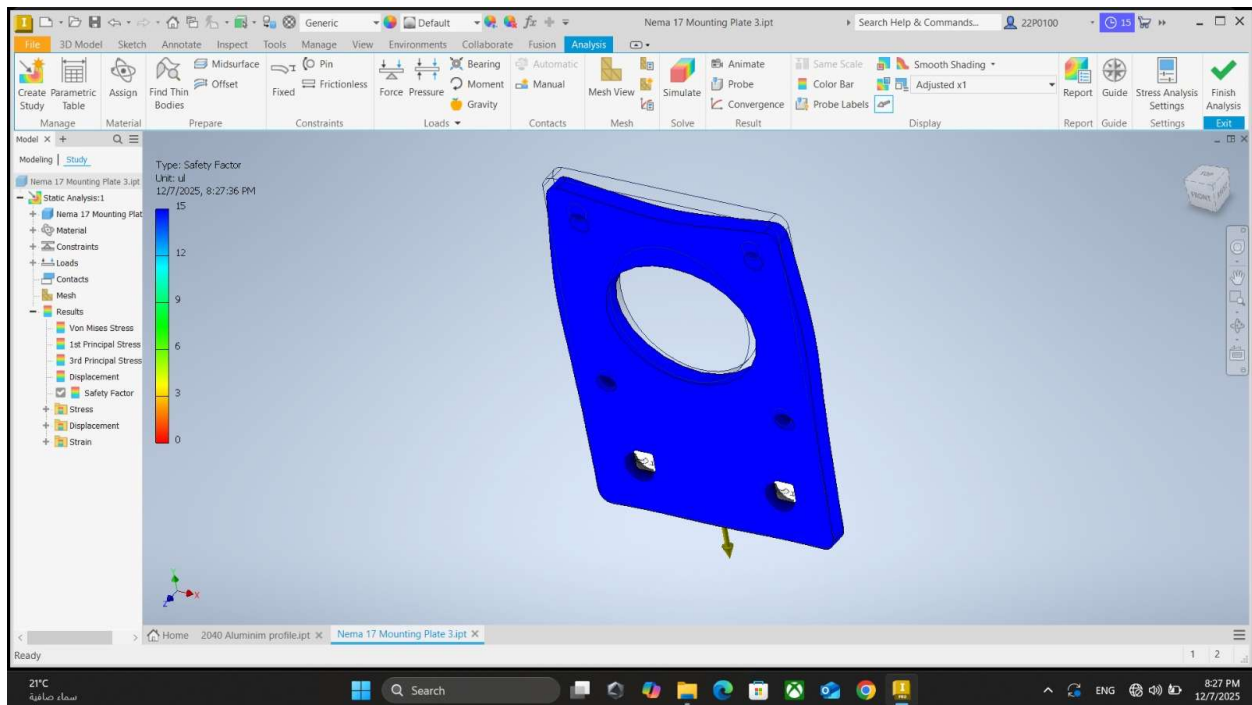
Figure 37 Wire Labelling

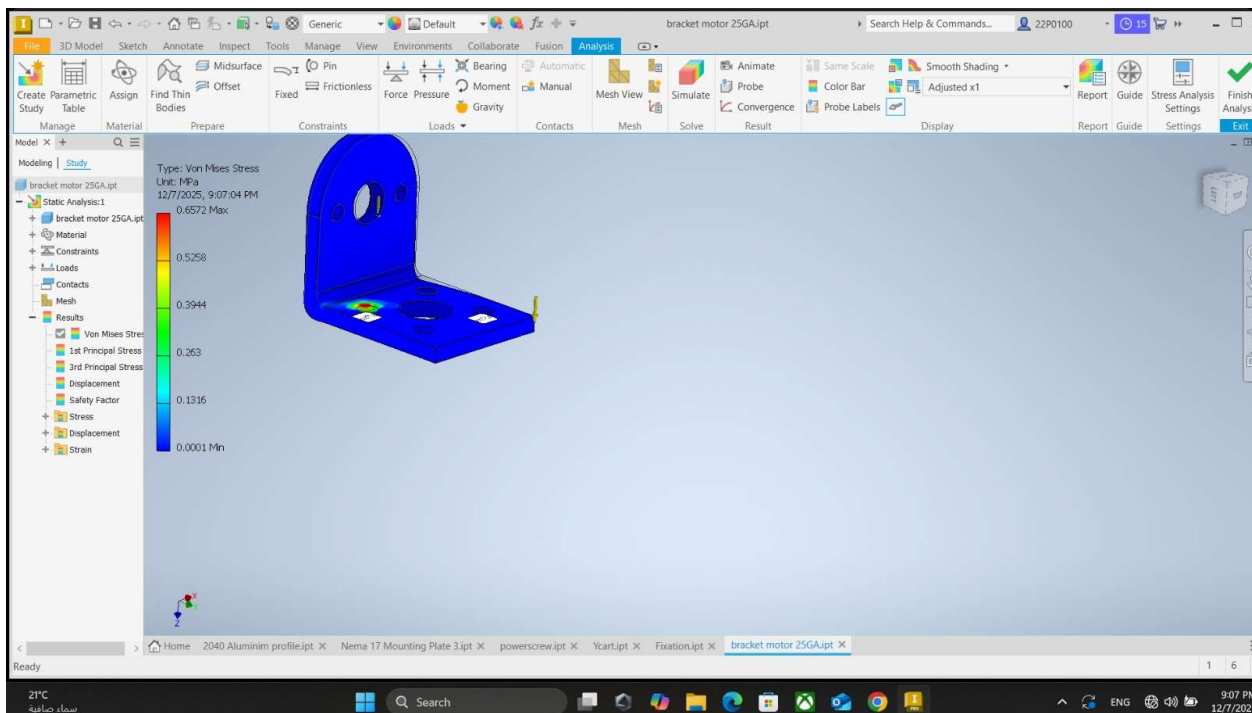
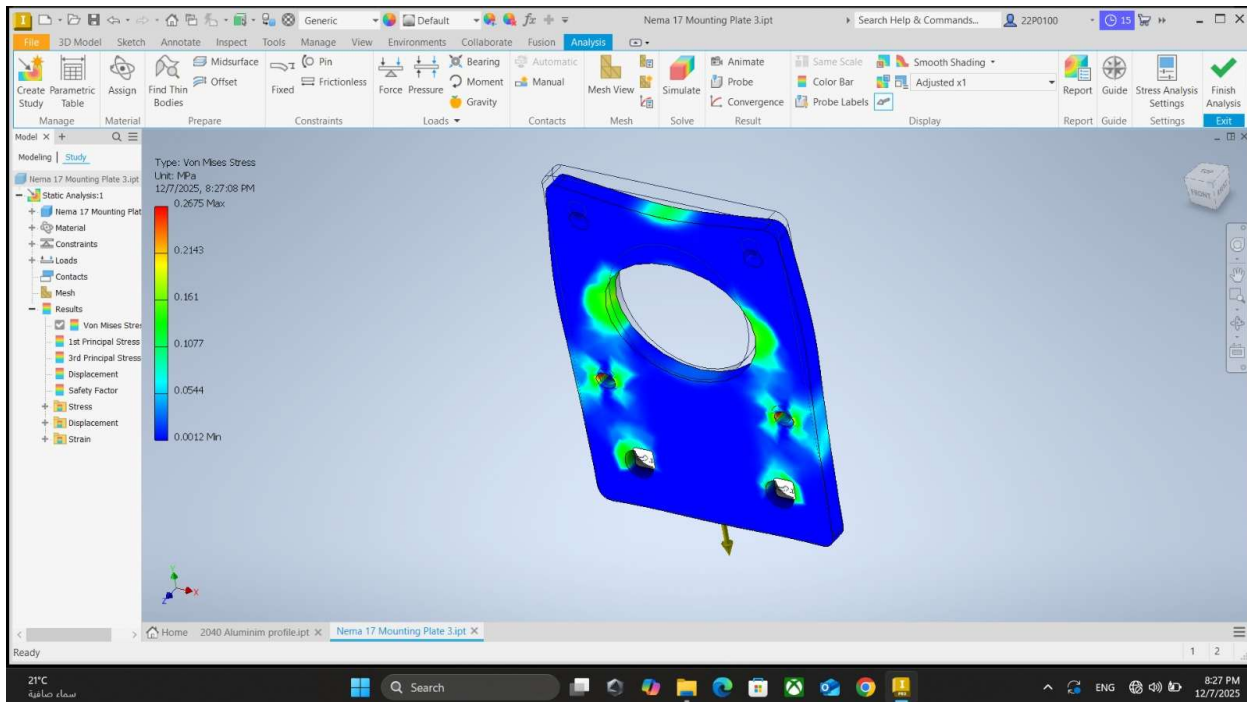
23 Stress Analysis on Critical Components

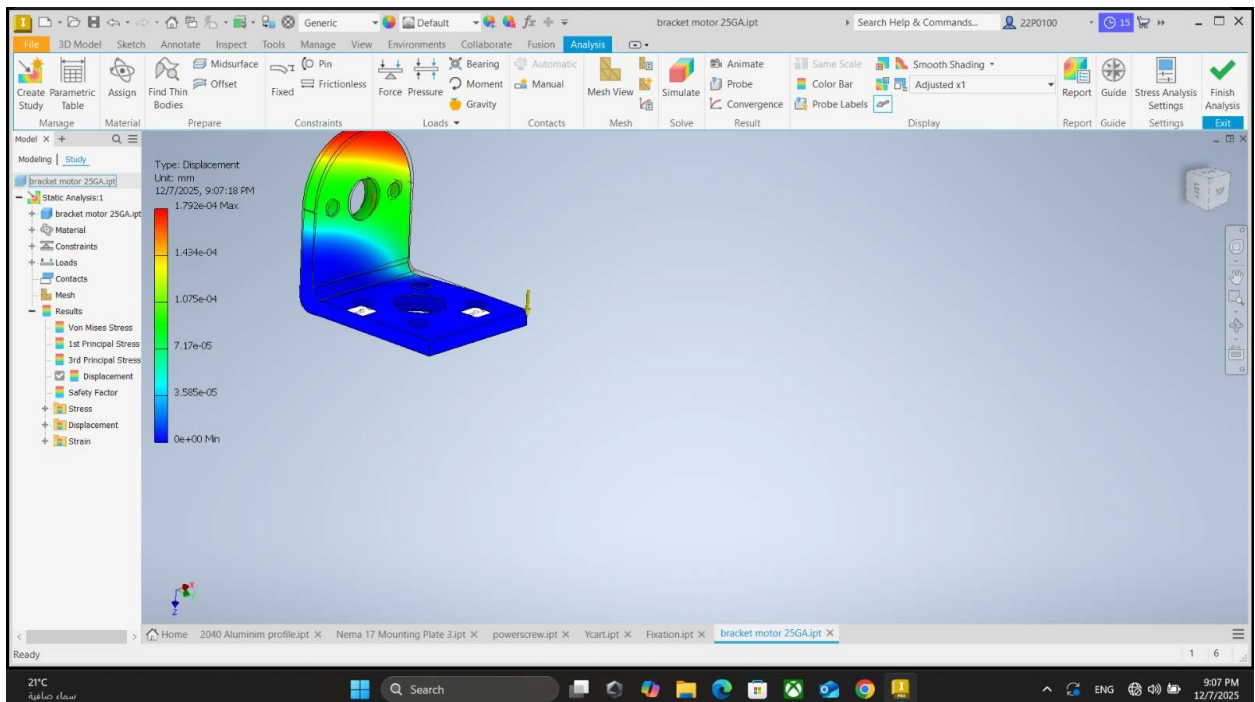
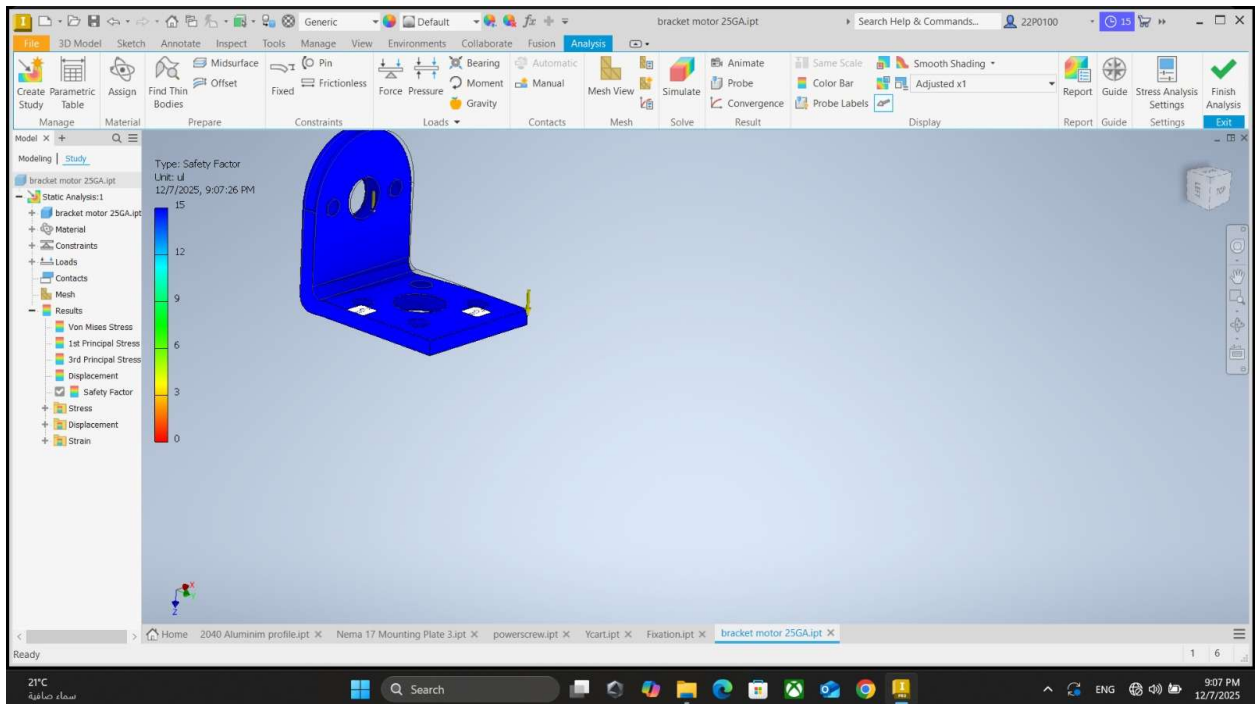
Finite Element Analysis (FEA) was performed on the critical structural components of the system to evaluate their mechanical integrity under maximum expected loading. The analysis included *Maximum Von Mises Stress*, *total displacement*, and *safety factor* evaluation. The results show that all stresses remain **well below the material yield limit**, and the maximum displacement is minimal and does not affect system accuracy. The **safety factor is significantly above 1.5–2**, confirming that the design can safely withstand operational loads with no risk of mechanical failure. These results validate that the structure is mechanically robust, stable, and capable of long-term operation without fatigue or deformation concerns.

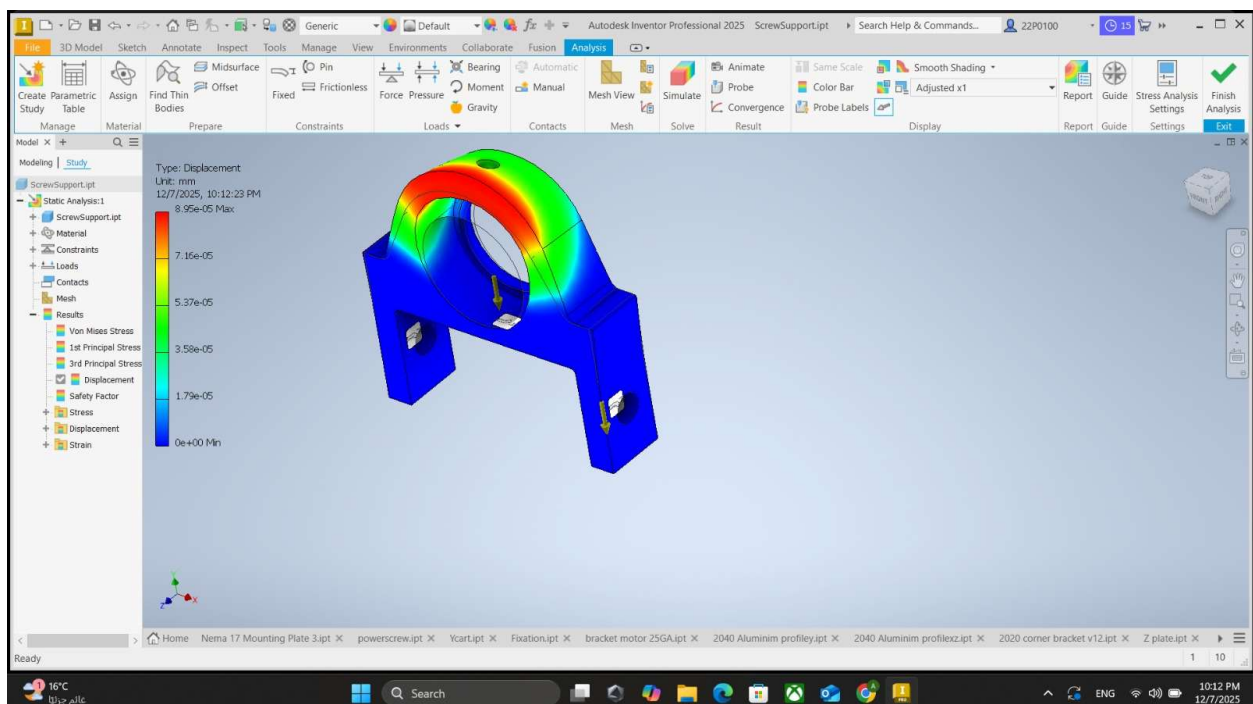
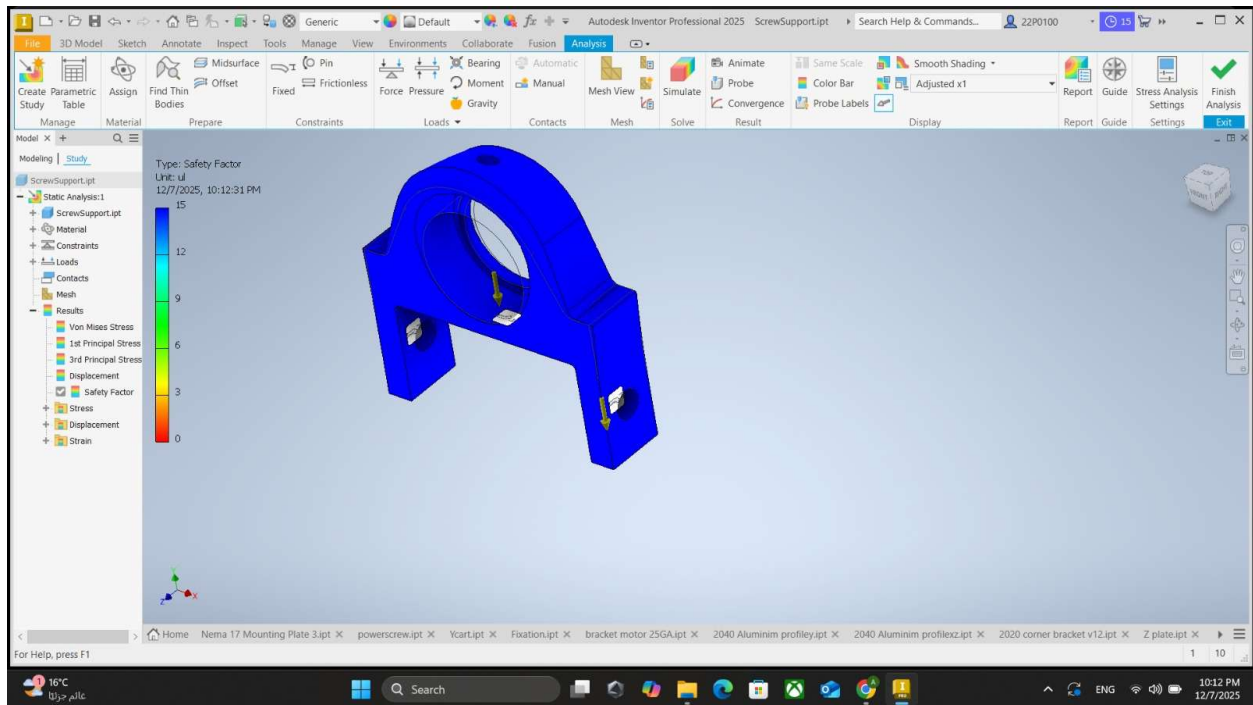


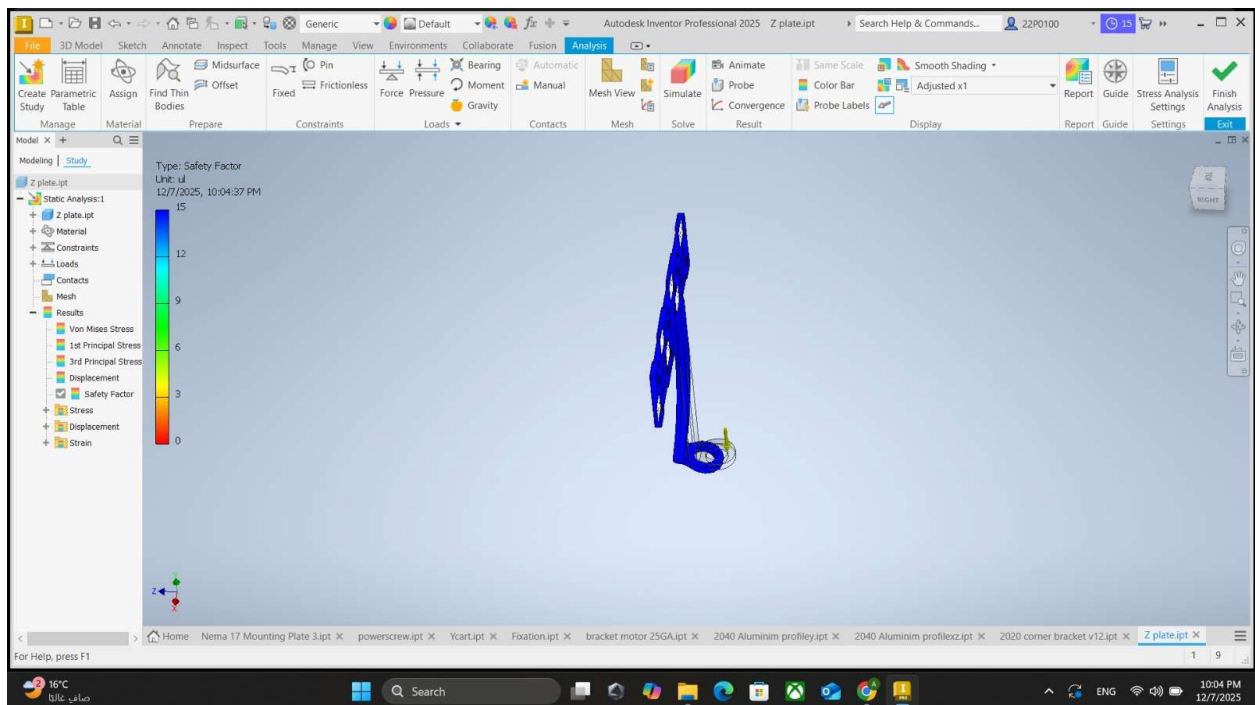
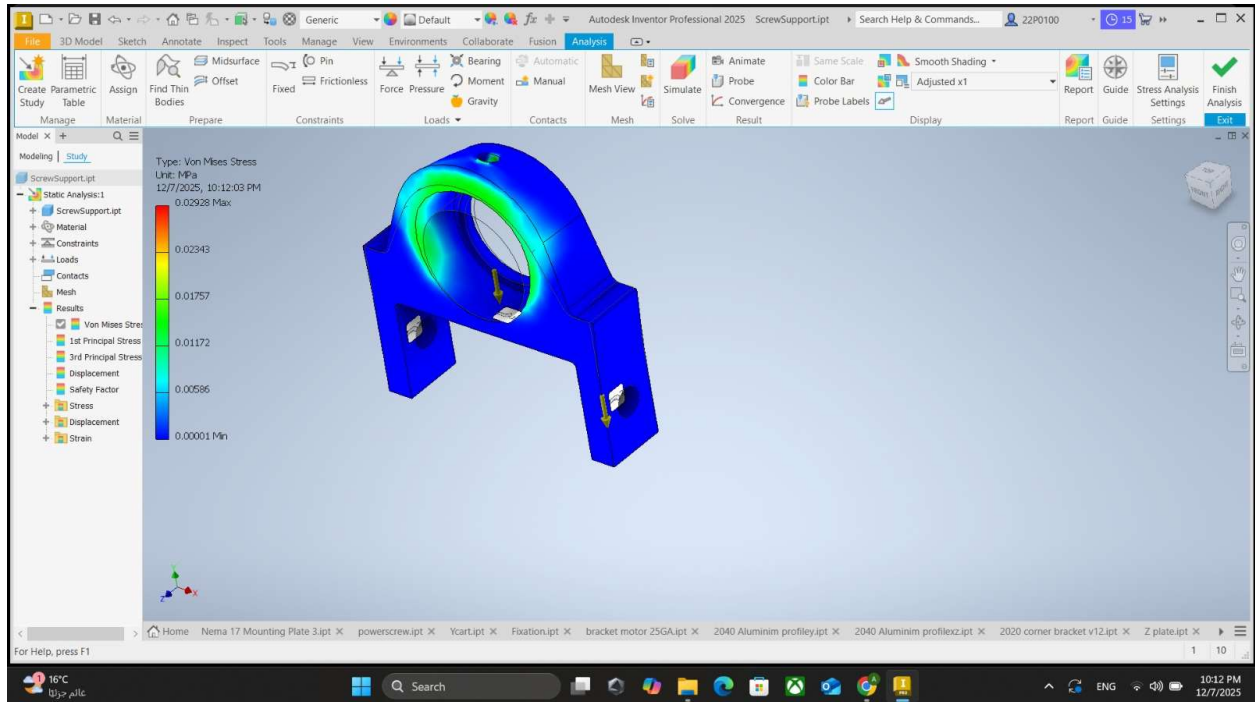


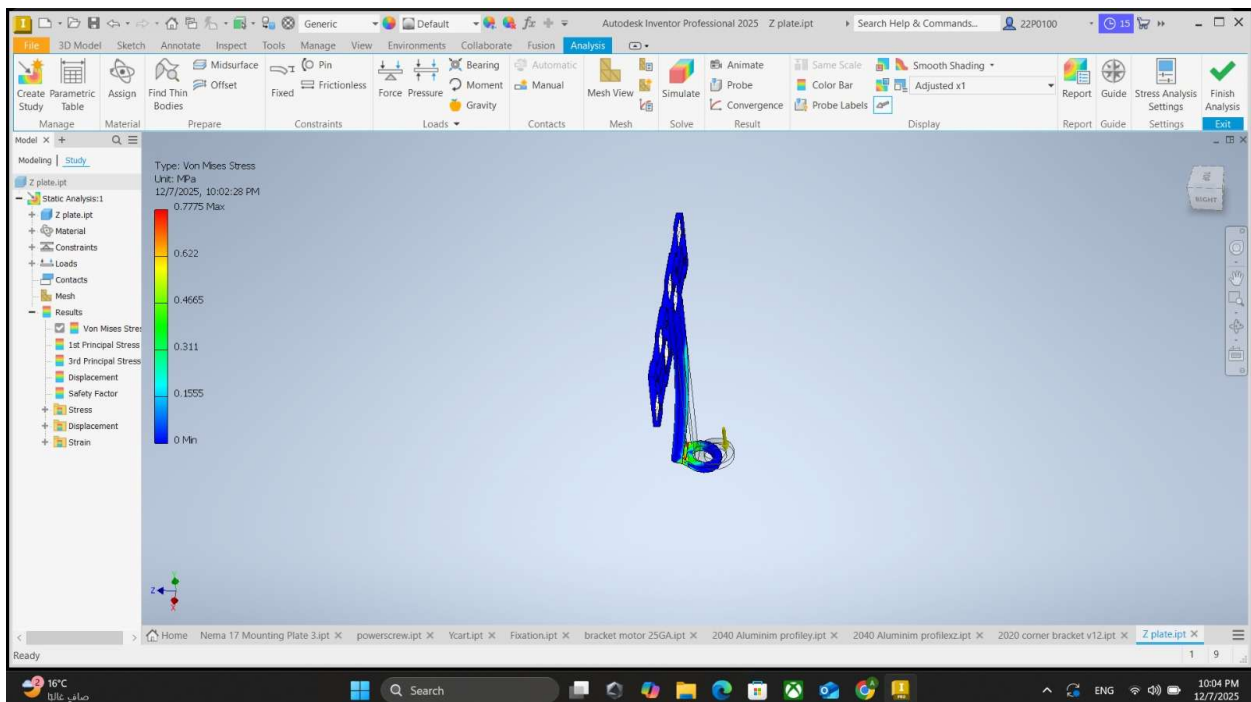
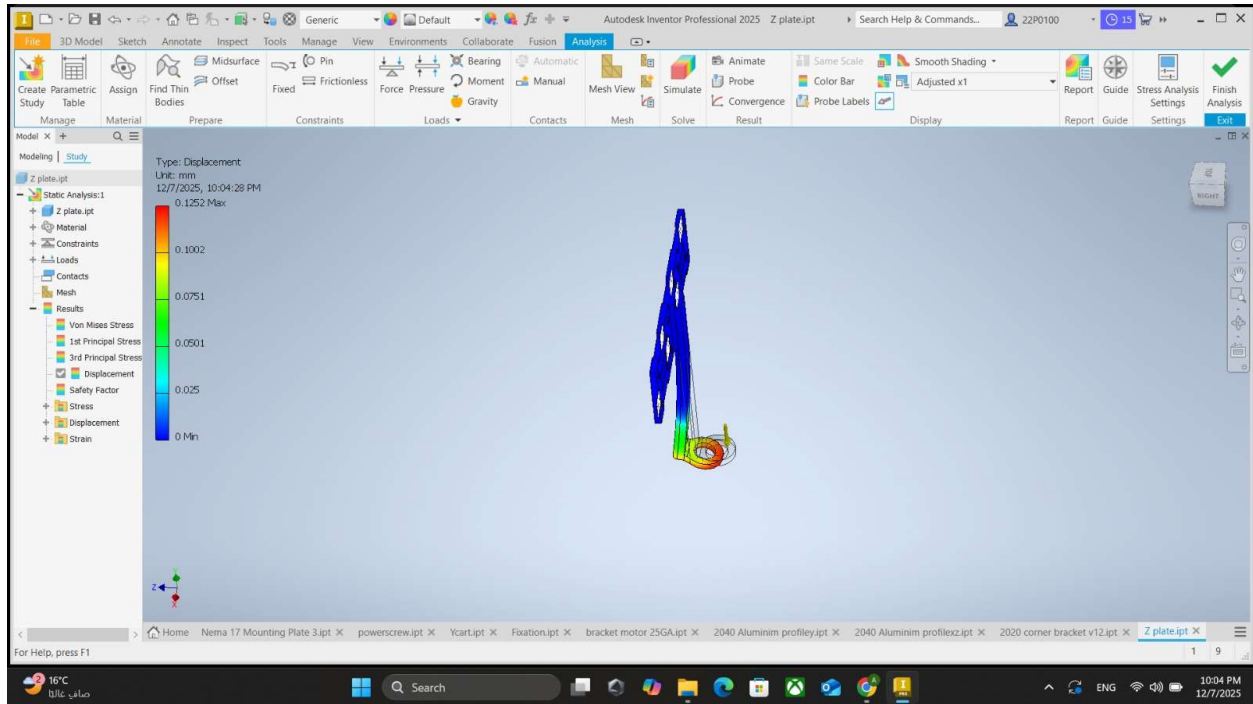


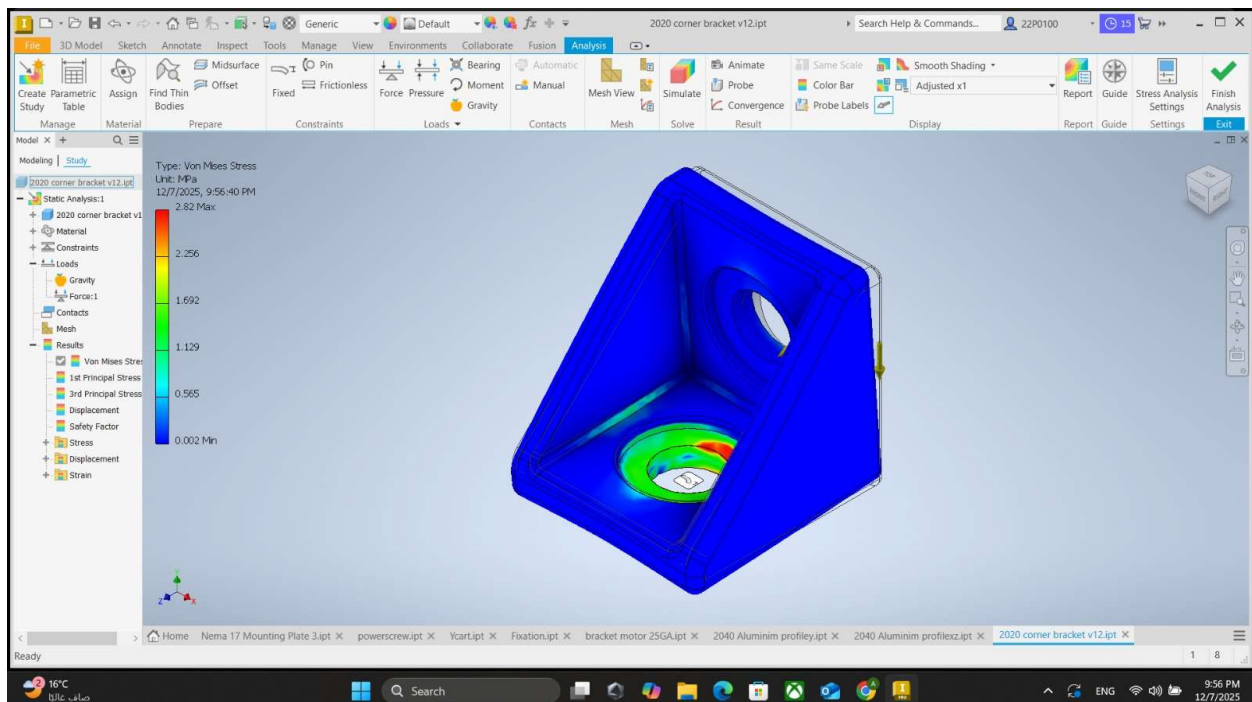
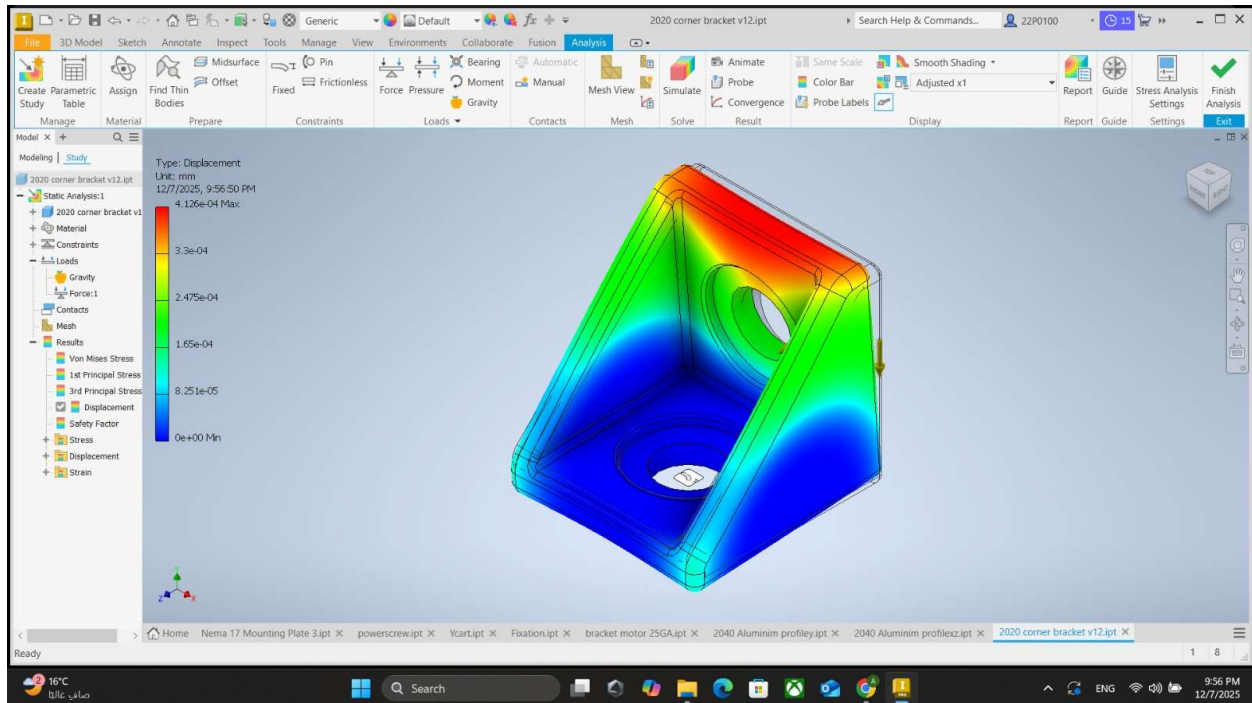


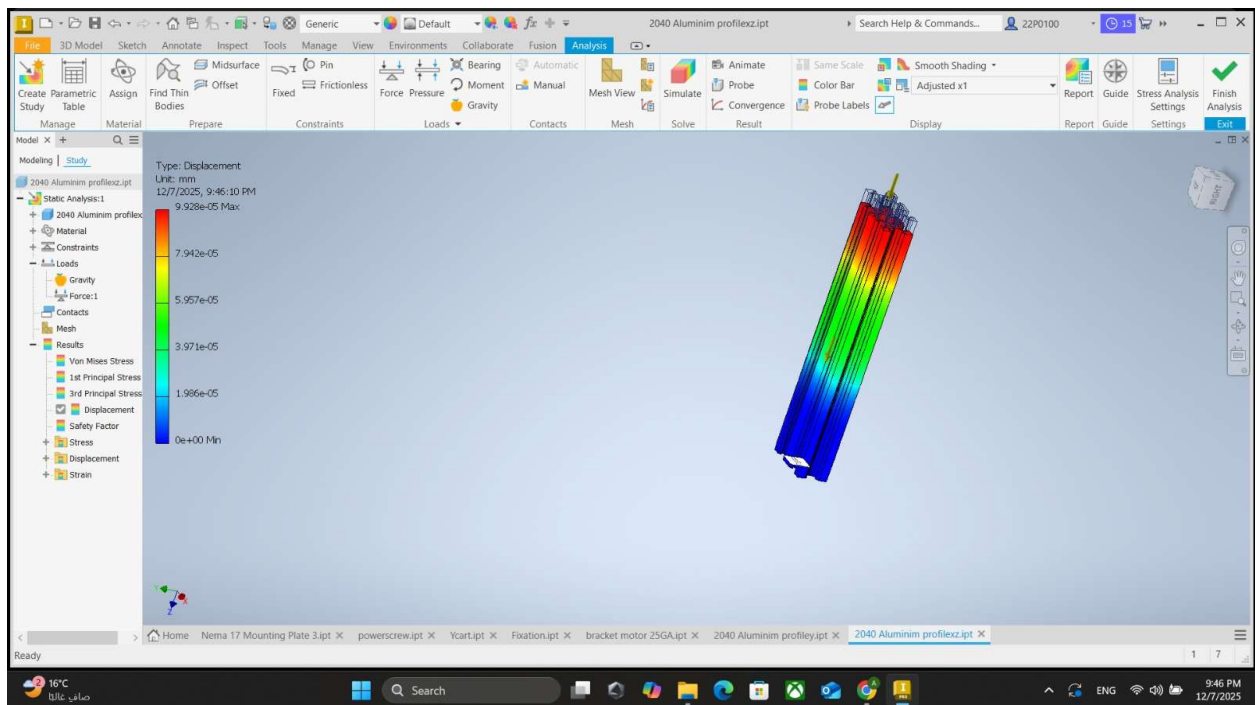
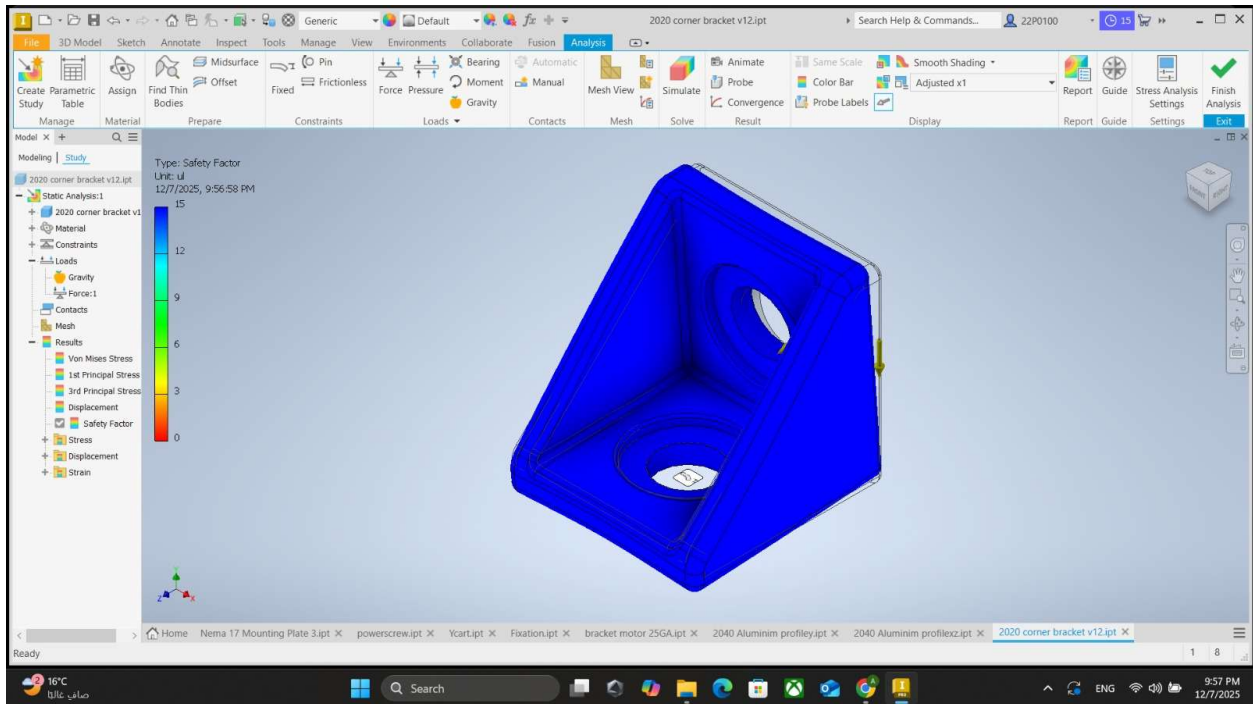


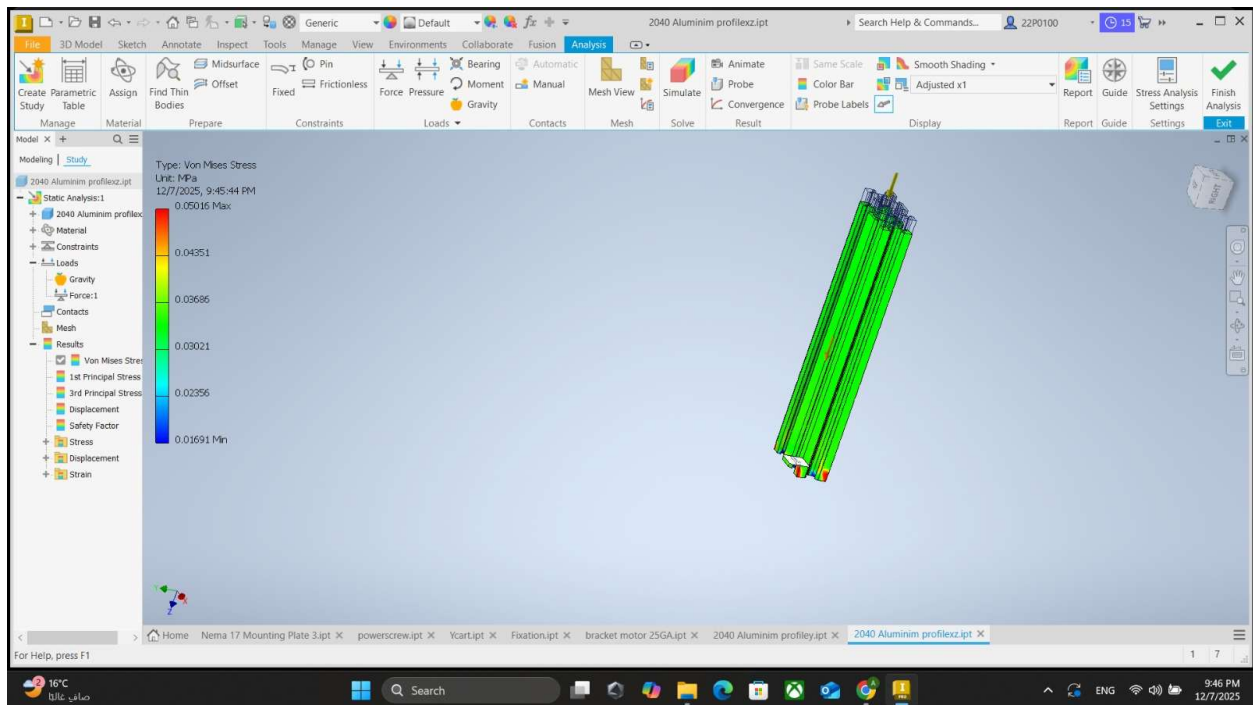
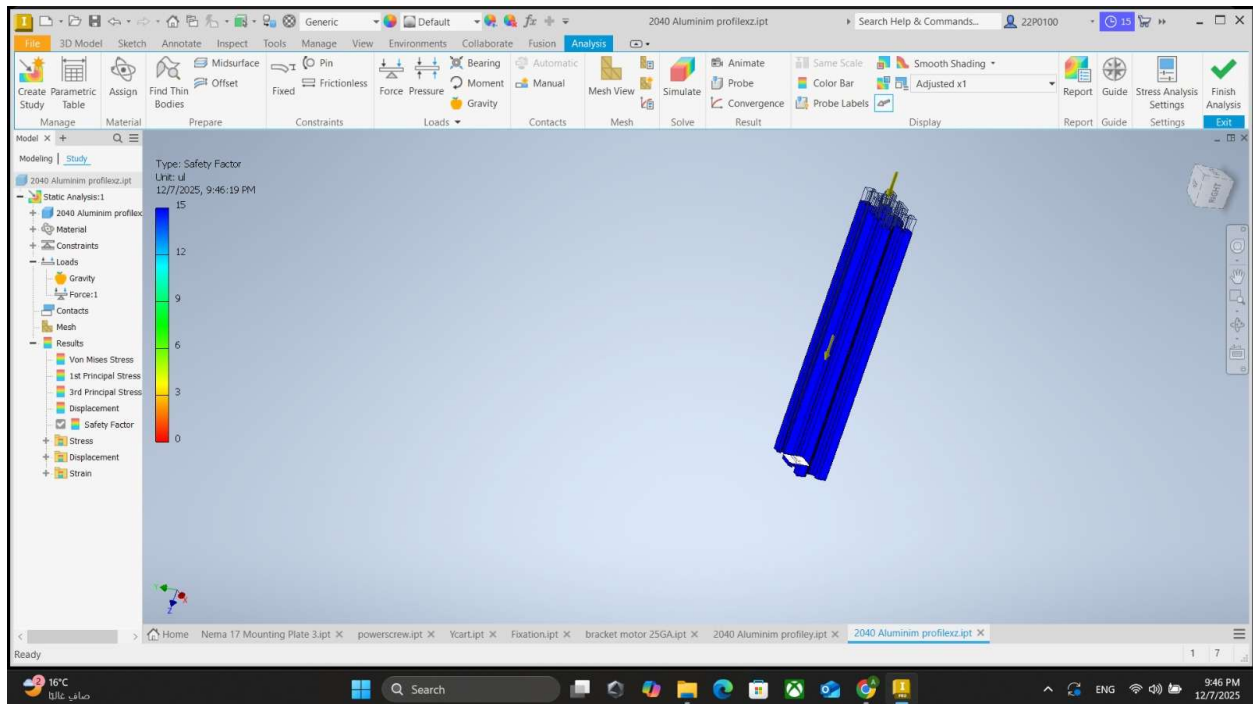


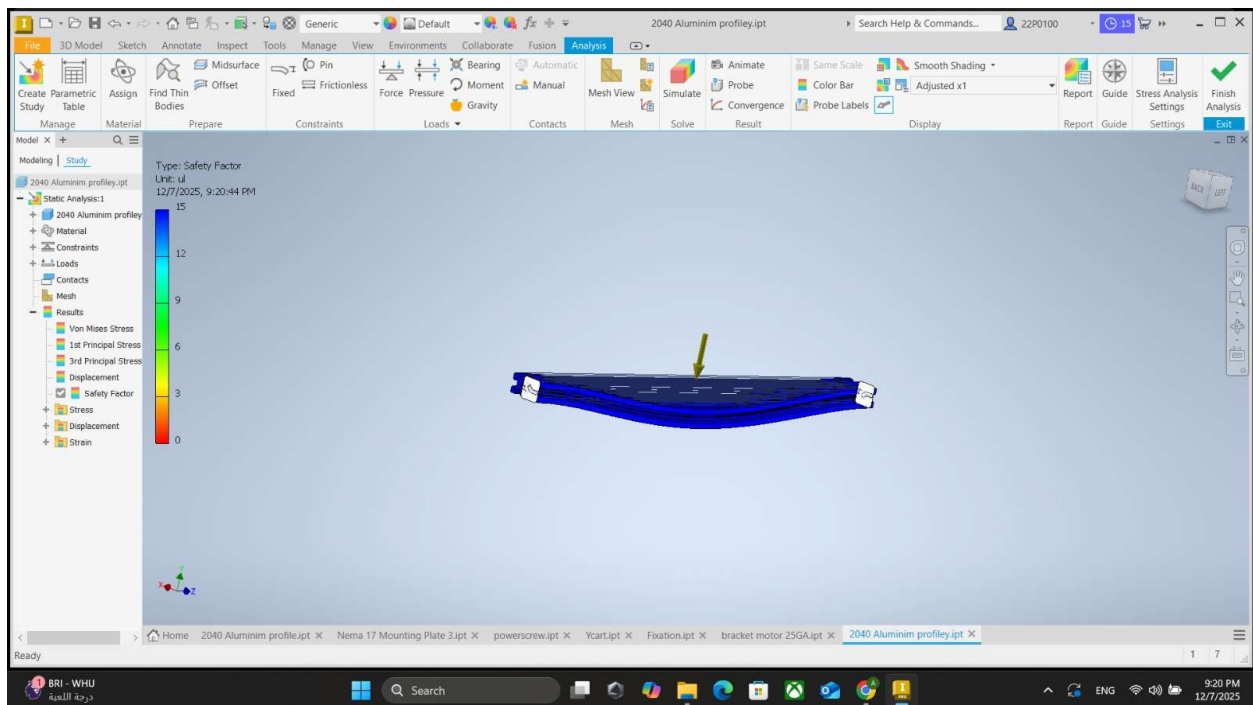
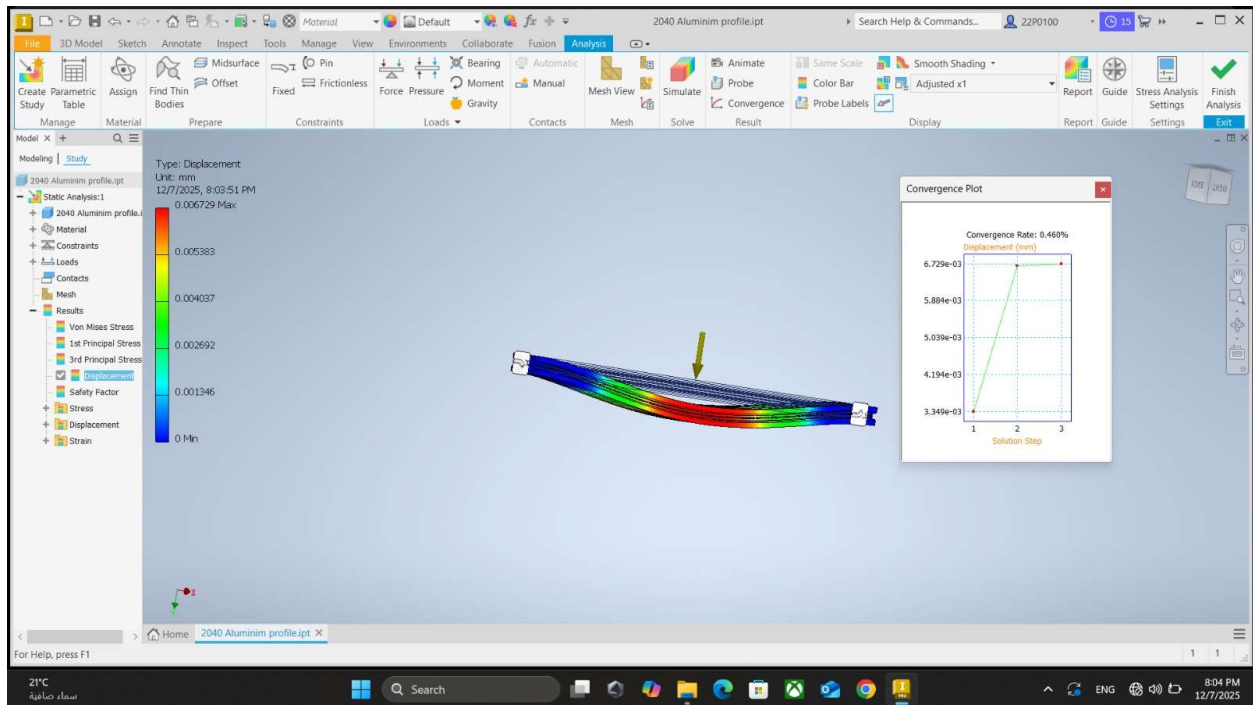


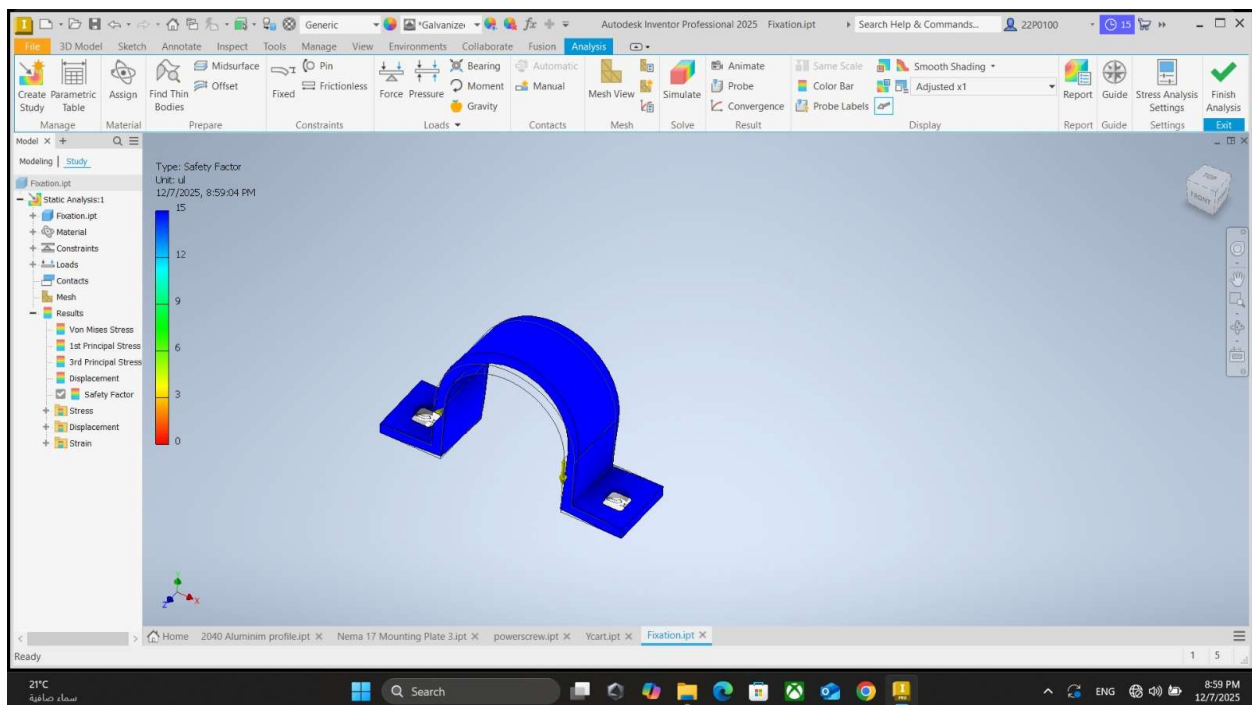
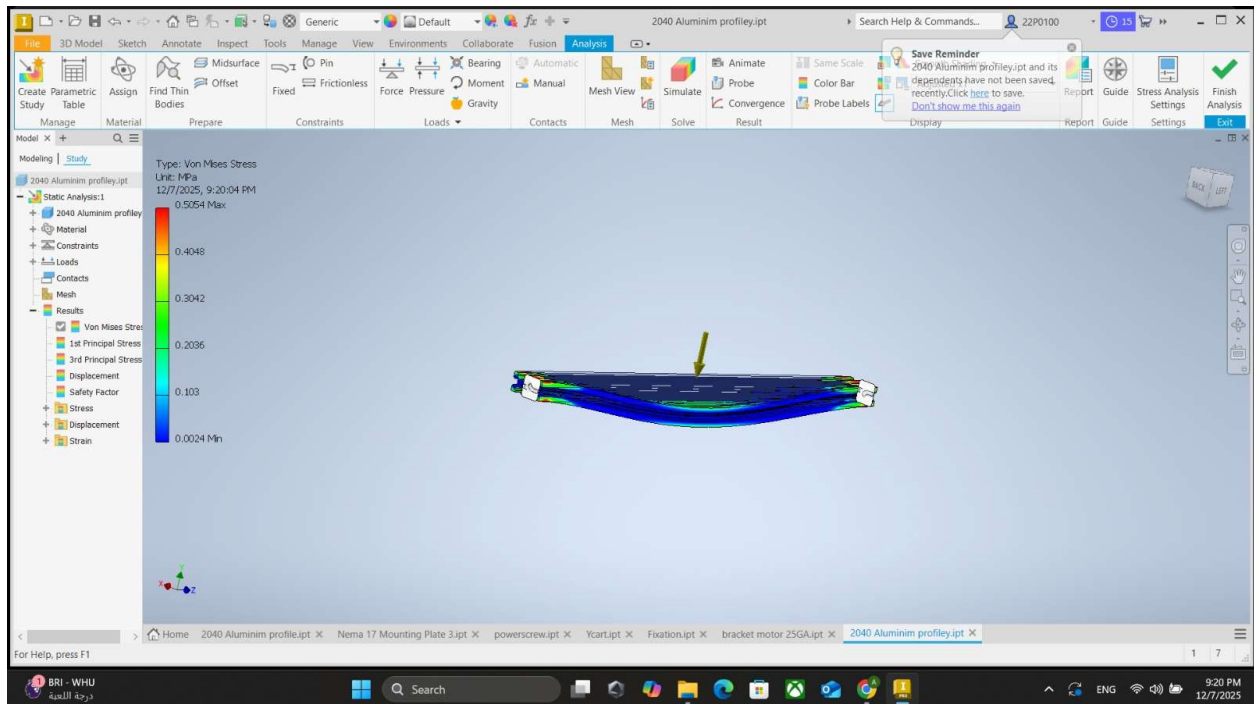


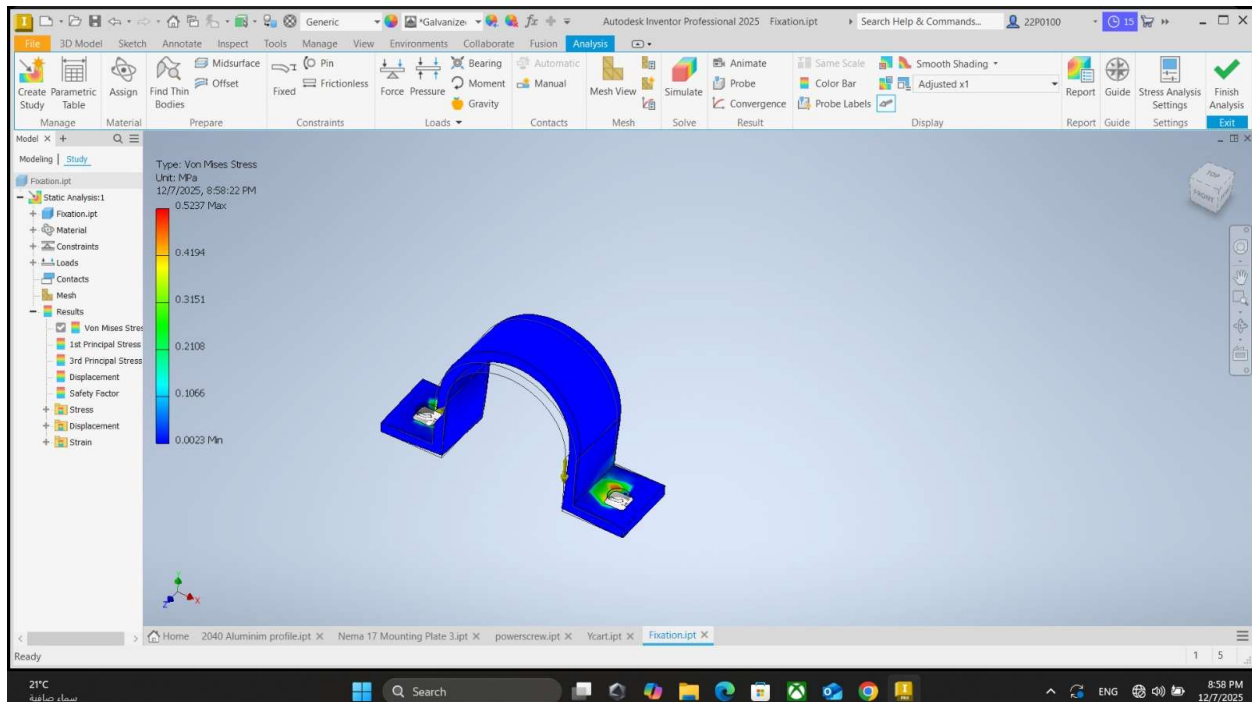
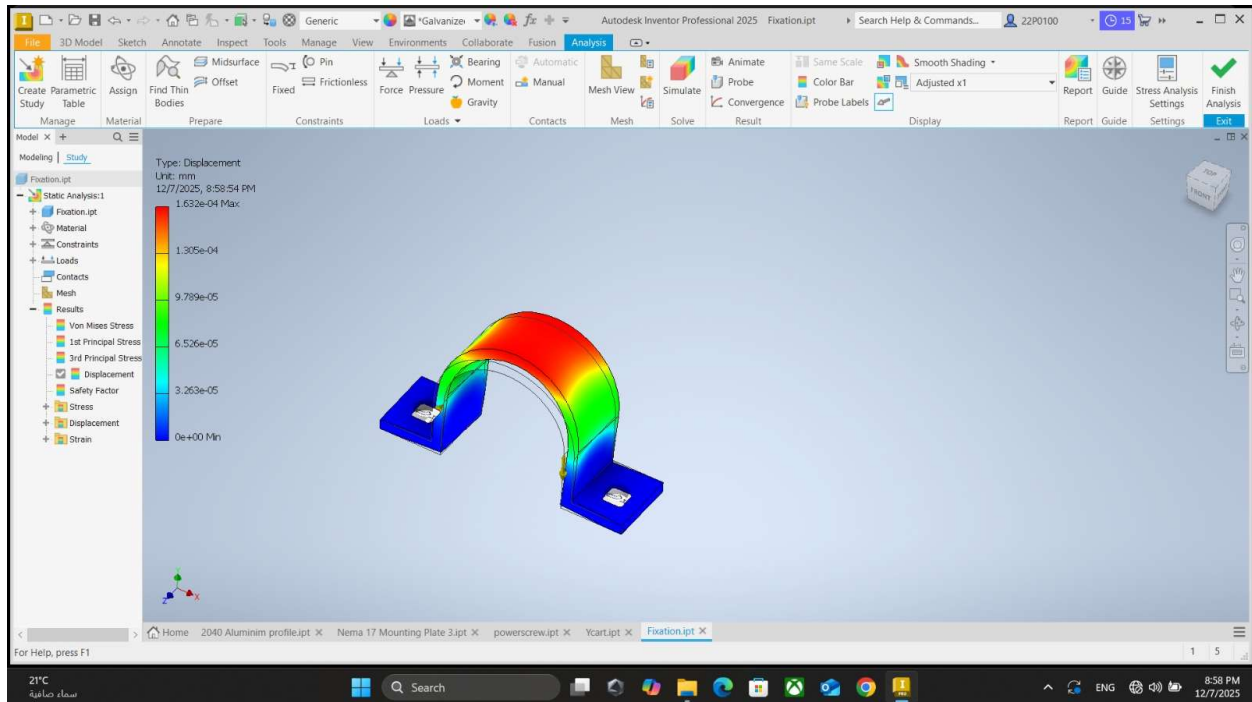


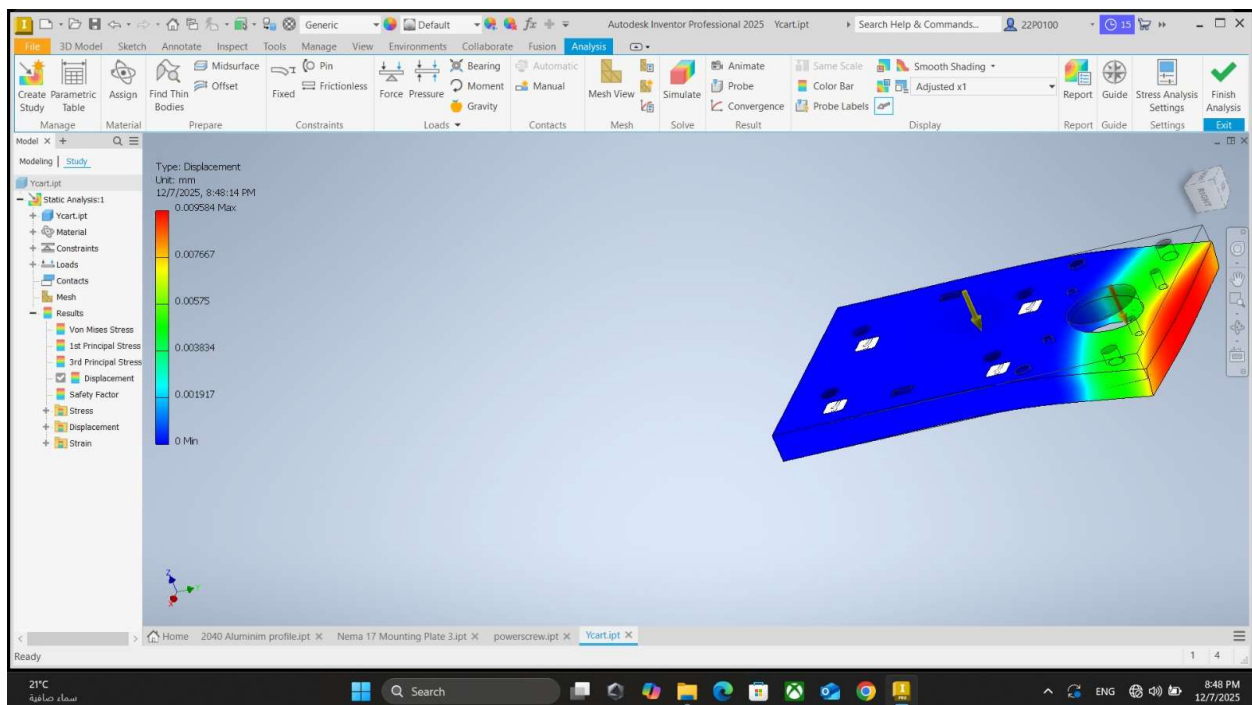
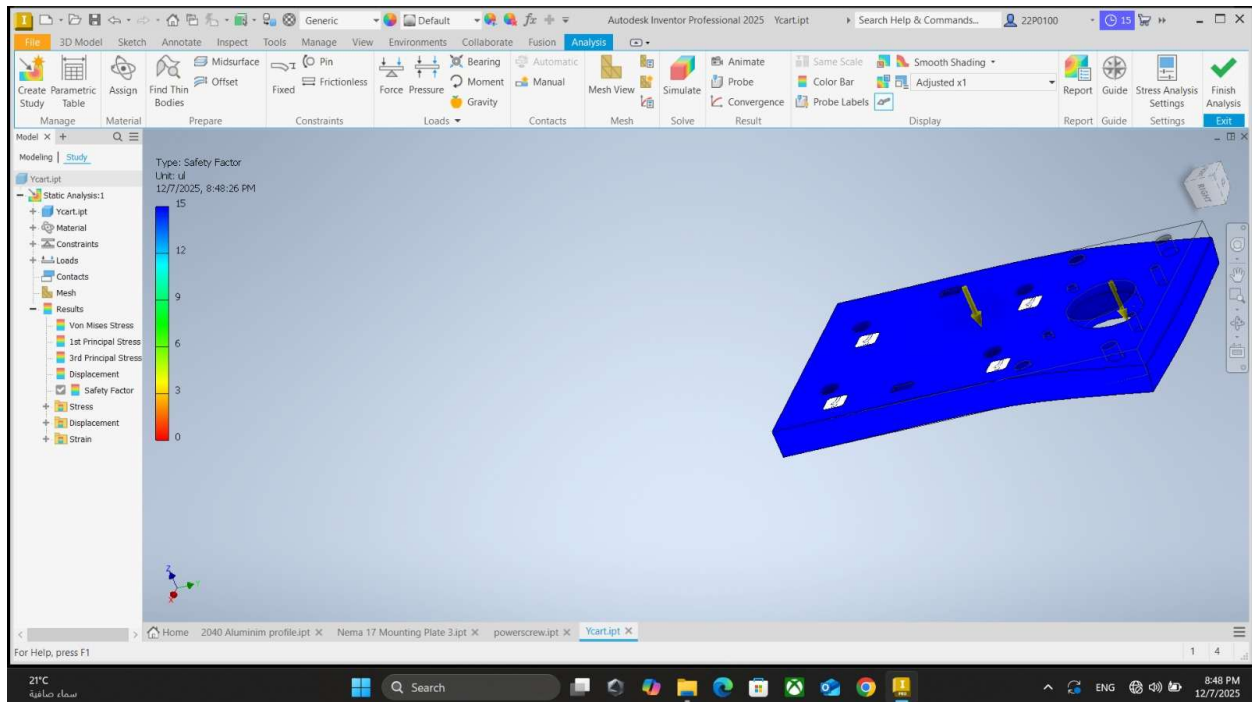


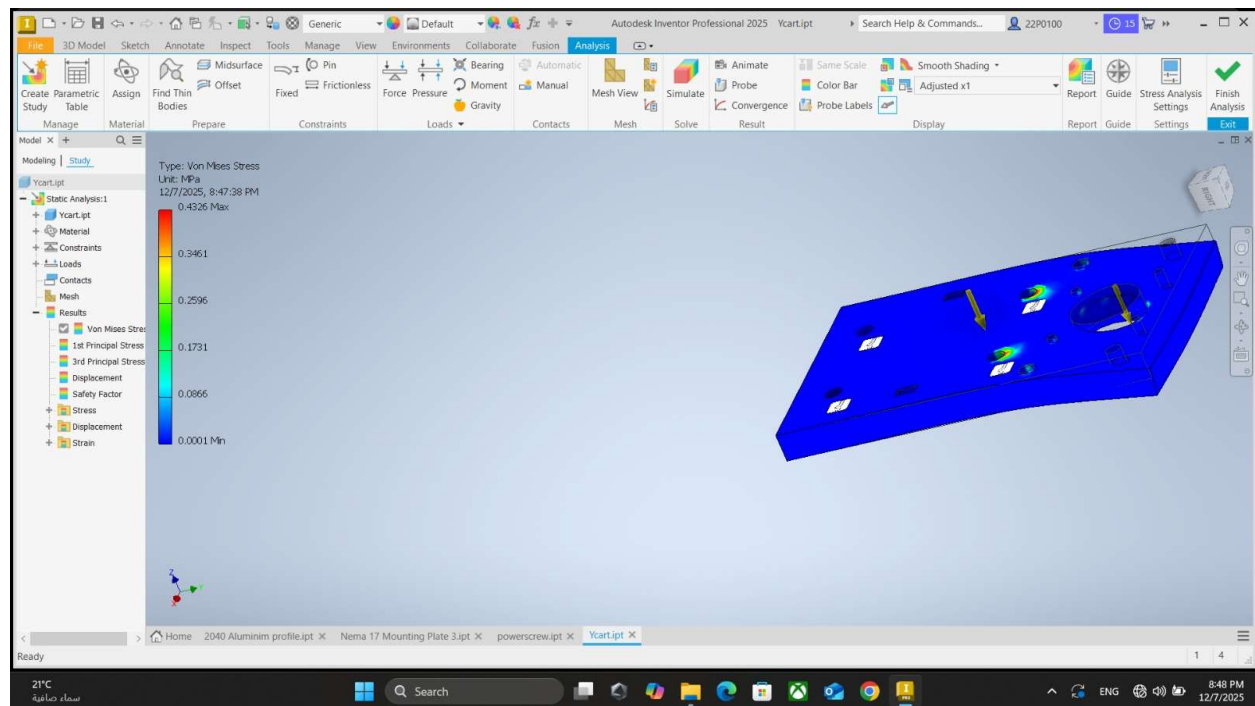




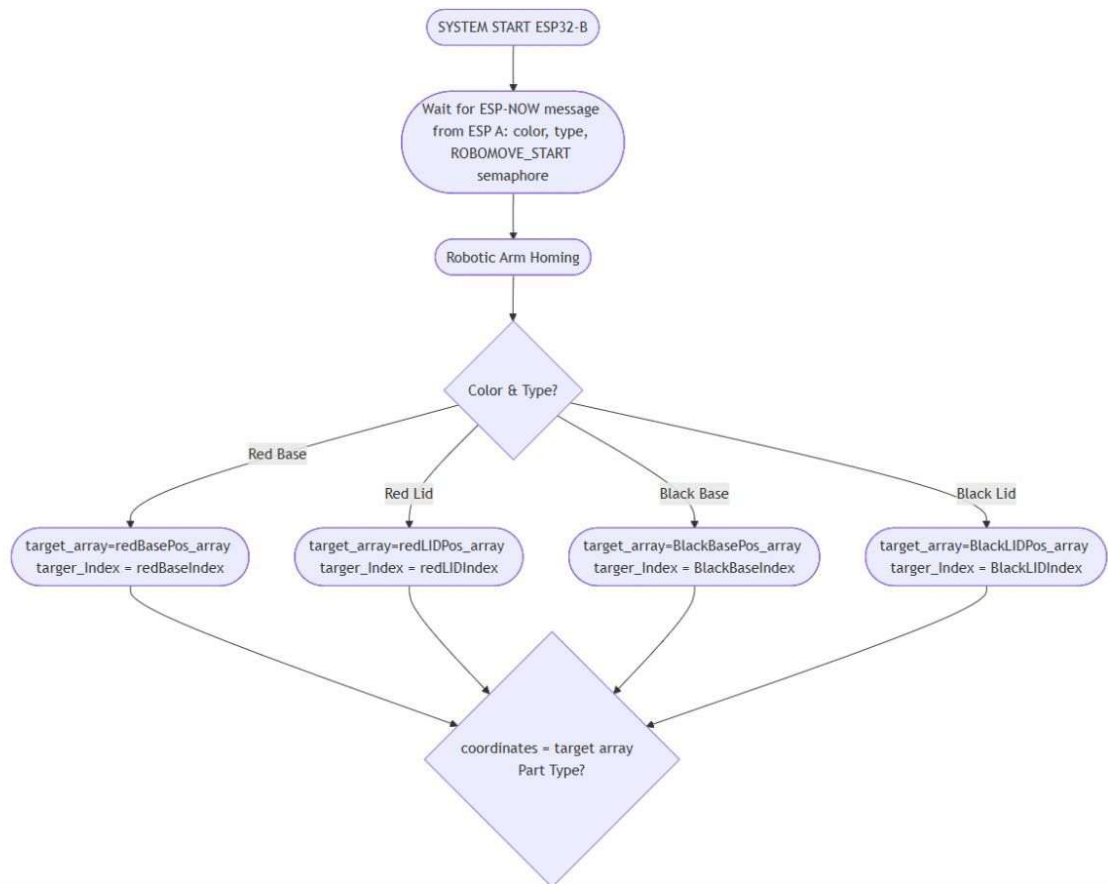


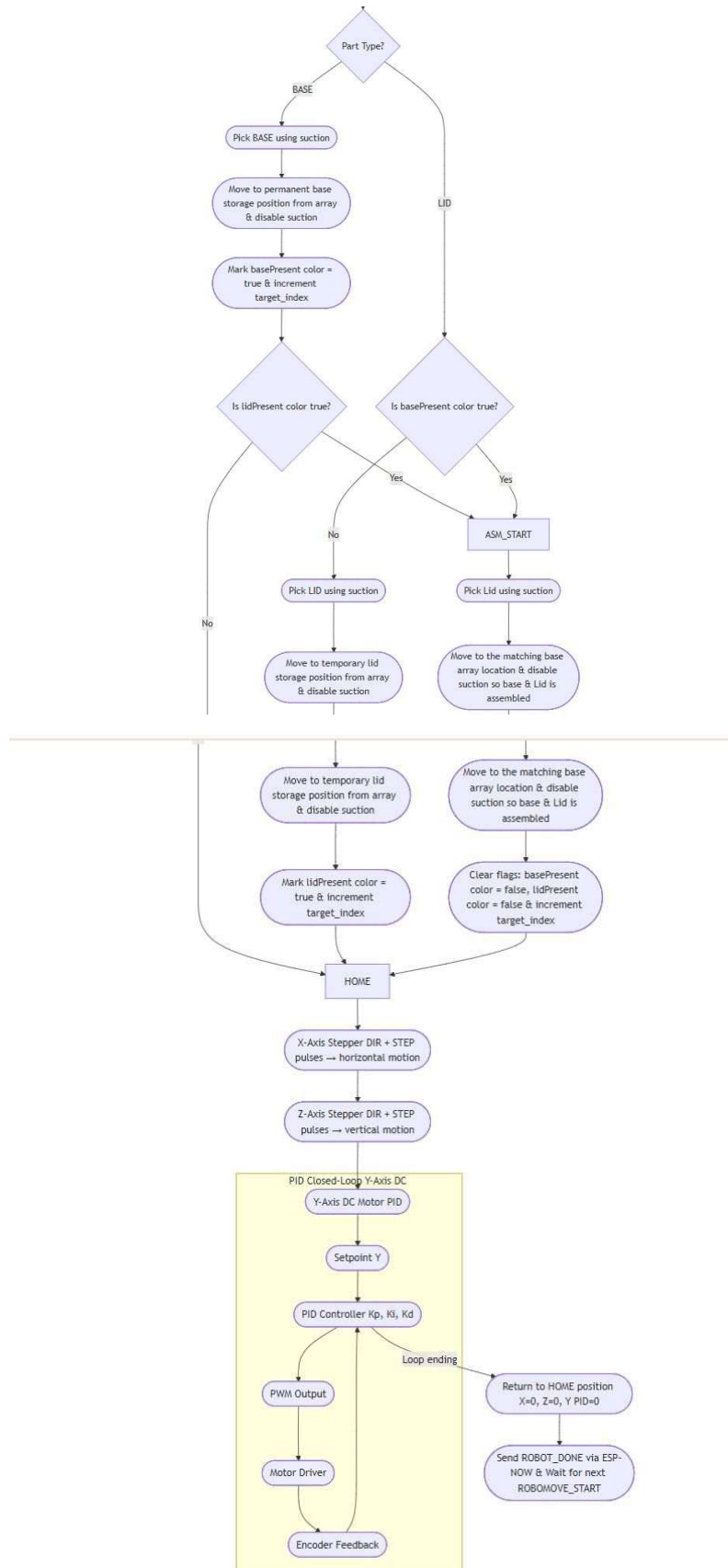


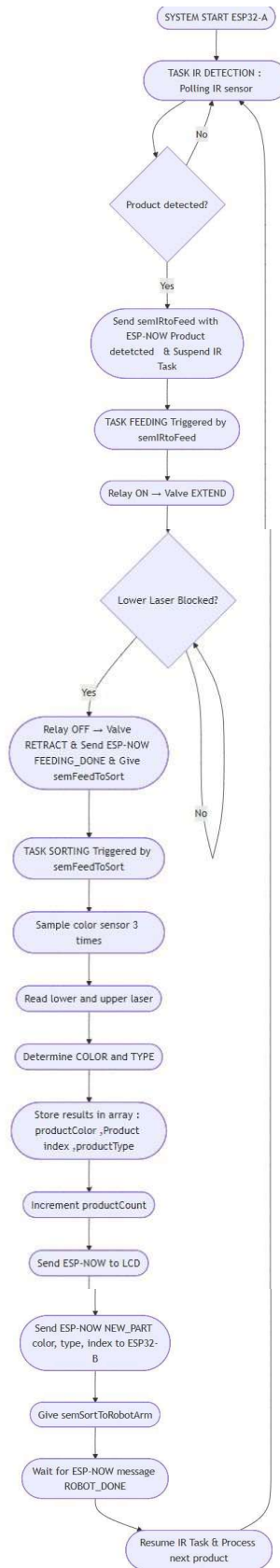




24 Flow charts







25 PID Controller Design and Implementation

25.1 PID Control Objective

The purpose of the PID controller is to achieve **accurate, stable, and smooth position control** of the Y-axis, which is driven by a DC motor with quadrature encoder feedback. The controller ensures that the axis reaches the desired position with **minimal steady-state error, acceptable transient response, and no oscillations**, satisfying the project PID evaluation criteria.

25.2 Control Structure

A **cascade control structure** was adopted, like the methodology presented in the PID laboratory experiment:

- **Outer loop (Position Control)**
Generates a velocity reference based on position error.
- **Inner loop (Velocity Control)**
Regulates motor speed using a PI controller.

This structure improves stability and dynamic performance compared to a single-loop PID.

25.3 Mathematical Model and Feedback

Encoder feedback provides real-time position measurements, which are converted from counts to millimeters using a calibrated scaling factor. Velocity is computed numerically using discrete differentiation based on the encoder count difference and fixed sampling time.

The control laws are implemented in discrete time due to the embedded nature of the system.

25.4 PID Tuning Methodology

The tuning process followed the same **step-by-step experimental approach** used in the PID laboratory:

1. **Proportional Position Gain (K_{p_pos})**
 - Increased gradually until the system showed fast response without oscillation.
 - Determines how aggressively the axis moves toward the target position.

2. Velocity Loop Proportional Gain (K_p_{vel})

- Tuned to provide responsive speed tracking.
- Prevents sluggish behavior during motion.

3. Velocity Loop Integral Gain (K_i_{vel})

- Added to eliminate steady-state velocity error caused by friction and load variations.
- Carefully limited to avoid windup and oscillations.

Final gains were selected experimentally to achieve a balance between speed, accuracy, and stability.

25.5 Anti-Windup and Dead-Zone Compensation

To improve real-world performance, practical enhancements were added:

- **Anti-windup logic** prevents integral accumulation when the PWM output saturates.
- **Dead-zone compensation** ensures motor movement despite static friction, especially at low velocities.
- **Stall detection** stops the motor safely if commanded motion does not result in encoder movement.

These additions significantly increase robustness compared to an ideal lab setup.

25.6 Real-Time Implementation Using FreeRTOS

The PID controller runs inside a **dedicated FreeRTOS task** with a fixed sampling period, ensuring deterministic behavior and consistent timing. Separating the PID task from motion sequencing tasks improves modularity and prevents control jitters.

25.7 Experimental Results and Performance

The implemented PID controller achieved:

- Smooth motion with **no oscillations**
- **Negligible steady-state error**
- Stable operation under continuous motion
- Reliable performance during pick-and-place and assembly operations

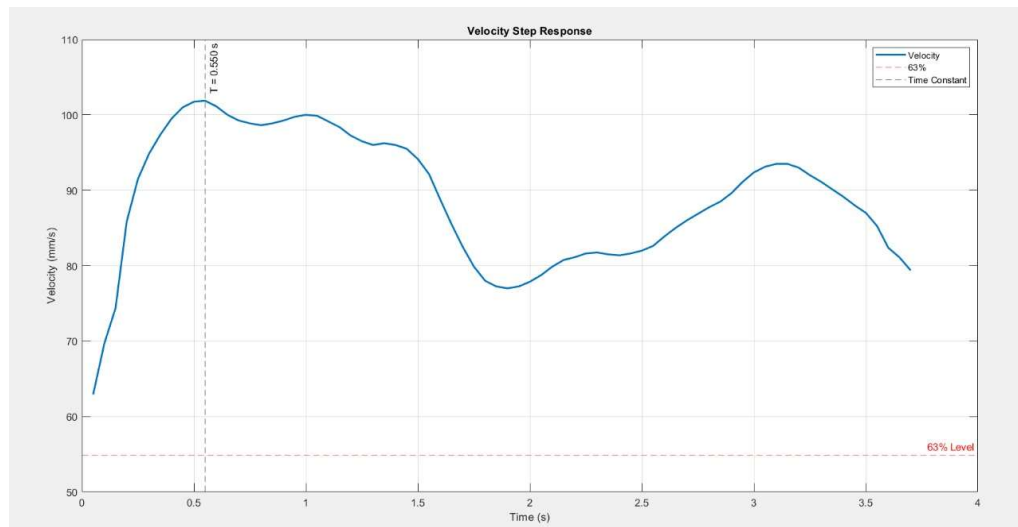


Figure 38 forward dc

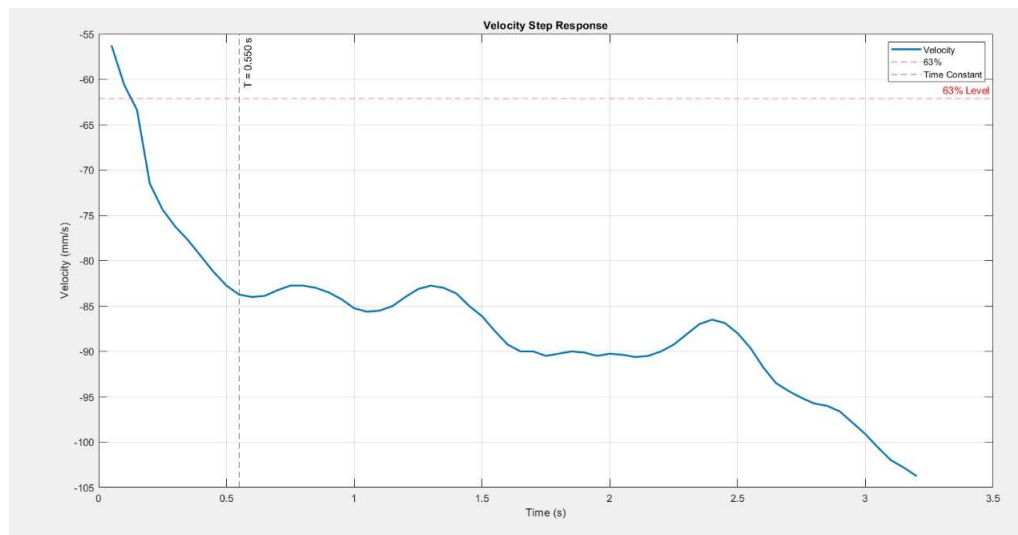


Figure 39 reverse dc

26 Software Implementation – ESP32 (1)

26.1 Overview of ESP32 (1) Role

ESP32 (1) is responsible for **product detection, feeding control, sorting decision, and communication**.

It operates in **real time** using **FreeRTOS**, controls the **pneumatic feeder**, reads **laser and color sensors**, and transmits sorting results to the second ESP32 via **ESP-NOW**. Local feedback is provided through an **I2C LCD display**.

26.2 RTOS Task Architecture

The software is structured into **three main FreeRTOS tasks**, each responsible for a specific subsystem:

Task Name	Function
IR Task	Detects product presence
Feeding Task	Controls pneumatic feeding
Sorting Task	Classifies product (height + color)

This task separation improves modularity, responsiveness, and fault isolation.

26.3 Inter-Task Synchronization Using Semaphores

Binary semaphores are used to control task execution order:

```
SemaphoreHandle_t semIRtoFeed;
SemaphoreHandle_t semFeedToSort;
```

- `semIRtoFeed` activates the feeding process after product detection
- `semFeedToSort` activates sorting after feeding completion

This ensures **strict sequential execution** without blocking the system.

26.4 Product Detection (IR Task)

The IR task continuously polls the infrared sensor to detect product presence. Once a product is detected, the task activates the feeding sequence and suspends itself to prevent false retriggering during feeding.

```
void IR_Task(void *pv) {  
    if (!irState && lastIRState) {  
        xSemaphoreGive(semIRtoFeed);  
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);  
    }  
}
```

26.5 Feeding Mechanism Control

The feeding task controls a **5/2 pneumatic solenoid valve** using a relay. The cylinder extends until the lower laser sensor is triggered, then retracts automatically.

```
digitalWrite(VALUE_PIN, HIGH);  
  
while (digitalRead(LOWER_LASER_RX) != HIGH) {  
    vTaskDelay(pdMS_TO_TICKS(10));  
}  
  
digitalWrite(VALUE_PIN, LOW);  
xSemaphoreGive(semFeedToSort);
```

26.6 Height-Based Sorting Using Laser Sensors

Two laser sensors are used to classify the product as **Base or Lid** based on beam obstruction.

```
uint8_t type = (lower == HIGH && upper == HIGH) ? PART_BASE : PART_LID;
```

26.7 Color Sorting Using TCS3200 Sensor

The TCS3200 color sensor measures RGB frequencies to classify products.

```
uint8_t color = readColor_TCS3200();
```

Inside the function:

- Red, green, and blue filters are selected sequentially
- Output frequency is measured
- Color decision is made using comparative thresholds

26.8 ESP-NOW Communication Between Controllers

After classification, the product data is sent wirelessly to ESP32 (2) using **ESP-NOW**.

```
typedef struct {  
    uint8_t color;  
    uint8_t type;  
} SortPacket;  
  
esp_now_send(receiverMac, (uint8_t *)&packet, sizeof(packet));
```

26.9 LCD Feedback and User Interface

The LCD provides immediate feedback for the operator.

```
void updateLCD(uint8_t color, uint8_t type) {  
    lcd.print("Color: ");  
    lcd.print(color == COLOR_RED ? "RED" : "BLUE");  
}
```

26.10 System Robustness and Safety

- IR task is suspended during feeding
- Laser sensors prevent over-extension
- Tasks run independently under RTOS
- No blocking delays are used

This ensures **stable continuous operation** even under high throughput.

27 Software Implementation – ESP32 (1)

27.1 System Overview

The presented code represents the **RTOS-based control software** running on the second ESP32 responsible for **assembly logic, motion sequencing, and axis coordination**. The system follows an **activity-based architecture**, where independent tasks (homing, supervision, PID control) operate concurrently under **FreeRTOS**, ensuring deterministic behavior and safe coordination between mechanical subsystems.

The software is designed to handle **all corner cases**, such as asynchronous arrival of parts, temporary storage, motion conflicts, and mechanical stall conditions.

27.2 FIFO Assembly Logic and Part Management

27.2.1 Data Structures and System Parameters

```
#define MAX_BASES 4
#define MAX_LIDS 4
#define ASSEMBLY_OFFSET 5
```

These parameters define **maximum storage capacity** and a fixed **assembly offset** to ensure correct vertical placement between the base and lid. Limiting storage size prevents memory overflow and enforces predictable system behavior.

```
typedef struct {
    int x;
    int y;
    int z;
} Position;
```

The Position structure represents **absolute machine coordinates**, allowing precise and repeatable motion planning across all axes.

```
enum ProductColor { RED, BLUE };
enum PartType { BASE, LID };
```

Enumerations are used instead of raw integers to **improve code readability, reduce logical errors**, and clearly distinguish product variants.

27.3 Storage Locations and FIFO Queues

```
Position redBasePos[MAX_BASES];  
Position blueBasePos[MAX_BASES];  
Position redLidTempPos[MAX_LIDS];  
Position blueLidTempPos[MAX_LIDS];
```

Separate arrays are assigned for each **color and part type**, ensuring physical separation of components and preventing incorrect assembly.

Temporary lid storage implements a **FIFO (First-In First-Out)** strategy, which is essential to:

- Preserve production order
- Avoid starvation of early-arriving parts
- Guarantee deterministic assembly behavior

27.4 Indexing and State Flags

```
int redLidWriteIndex, redLidReadIndex;  
bool redBasePresent, redLidPresent;
```

Indices and flags are used together to:

- Track storage availability
- Detect assembly readiness
- Prevent invalid access to empty queues

This approach ensures **safe operation even when parts arrive in arbitrary order**, a key corner case in automated assembly lines.

27.5 X-Axis: Dual Stepper Motor Control

The X-axis uses **two synchronized stepper motors** to ensure structural rigidity and parallel motion. Each motor is driven independently but commanded with identical step counts and timing.

Importance:

- Dual motors prevent racking and uneven load distribution.
- Software synchronization ensures precise linear motion without mechanical coupling.

The relative motion function converts millimeters to steps and executes synchronized stepping:

```
#define X_STEP_A 33
#define X_DIR_A 4
#define X_ENA_A 32

#define X_STEP_B 19
#define X_DIR_B 5
#define X_ENA_B 21

bool Arm_MoveX(float dx_mm) {
    if (!x_homed || arm_busy) return false;
    arm_busy = true;

    stepsRemainingX_A = labs(dx_mm * X_STEPS_PER_MM);
    stepsRemainingX_B = stepsRemainingX_A;

    bool dir = (dx_mm > 0) ^ X_DIR_INVERT;
    digitalWrite(X_DIR_A, dir);
    digitalWrite(X_DIR_B, dir);

    while (stepsRemainingX_A > 0 || stepsRemainingX_B > 0) {
        if (stepsRemainingX_A > 0) stepXA();
        if (stepsRemainingX_B > 0) stepXB();
        vTaskDelay(pdMS_TO_TICKS(1));
    }

    x_pos_mm += dx_mm;
    arm_busy = false;
    return true;
}
```

Design Significance:

- Uses **non-blocking RTOS delays** to avoid CPU starvation.
- arm_busy prevents conflicting commands from multiple tasks.
- Motion is safe-guarded by homing validation.

27.6 Y-Axis: Encoder-Based Closed-Loop PID Control

```
ESP32Encoder yEncoder;  
const float Y_COUNTS_PER_MM = 36.0;
```

Encoder-Based Feedback

This task runs continuously under RTOS and performs:

- Position error calculation
- Velocity reference generation
- PI velocity control
- Dead-zone compensation
- Stall detection and recovery

```
void Y_PID_Task(void *pv) {  
    long prevCount = yEncoder.getCount();  
    float vel_integral = 0;  
  
    for (;;) {  
        long position = yEncoder.getCount();  
        float vel_meas = (position - prevCount) / Y_TS;  
        prevCount = position;  
  
        float pos_error = TARGET_COUNTS - position;  
        float vel_ref = Kp_pos * pos_error;  
  
        float vel_error = vel_ref - vel_meas;  
        int pwm_cmd = Kp_vel * vel_error + Ki_vel * vel_integral;  
  
        setYMotor(pwm_cmd);  
        vTaskDelay(pdMS_TO_TICKS(Y_TS_MS));  
    }  
}
```

If motion is commanded but no encoder feedback is detected:

- The motor is stopped
- The integral term is reset
- A stall warning is issued

This prevents **motor overheating, mechanical damage, and loss of position**, significantly improving system safety.

27.7 Z-Axis: Vertical Stepper Control

The Z-axis motion is executed using relative displacement commands translated into step counts based on a calibrated steps-per-millimeter constant. This approach ensures accurate vertical positioning during pick-and-place operations. Motor enable control is applied to reduce power consumption and prevent unnecessary holding torque when the axis is idle.

A software interlock (`arm_busy`) is used to prevent simultaneous motion commands across multiple axes, ensuring mechanical safety and preventing collision or synchronization errors.

```
bool Arm_MoveZ(float dz_mm) {
    if (!z_homed || arm_busy) return false;

    stepsRemainingZ = labs(dz_mm * Z_STEPS_PER_MM);
    digitalWrite(Z_DIR, (dz_mm > 0));

    while (stepsRemainingZ > 0) {
        stepZ();
        stepsRemainingZ--;
        vTaskDelay(pdMS_TO_TICKS(1));
    }

    z_pos_mm += dz_mm;
    arm_busy = false;
    return true;
}
```

27.8 Homing Sequence and Reference Initialization

A unified homing routine initializes all three axes (X, Y, Z) in a controlled and safe order.

- **Y-axis** homing is performed first with the PID controller disabled to avoid control conflicts.
- **X-axis** homing is executed using two synchronized motors, each monitored by its own limit switch to ensure gantry alignment.
- **Z-axis** homing is executed last and only after X and Y axes are confirmed homed, preventing vertical motion over undefined positions.

This structured homing sequence guarantees a known mechanical reference, which is critical for repeatable and safe operation.

```
void Arm_HomeAll() {  
    y_pid_enabled = false;  
    while (digitalRead(Y_MIN_PIN) == LOW) setYMotor(Y_HOMING_PWM);  
    yEncoder.clearCount();  
    y_homed = true;  
    y_pid_enabled = true;  
}
```

27.9 Absolute Motion Abstraction

To simplify motion planning and improve readability, absolute position commands are converted internally into relative movements.

This abstraction layer allows higher-level logic to command motion using workspace coordinates without handling low-level motor steps directly.

Each axis updates a shared `currentPos` structure, which maintains the robot's absolute position in real time and enables coordinated multi-axis motion.

```
typedef struct {  
    float x;  
    float y;  
    float z;  
} Position;  
  
Position currentPos = {0, 0, 0};  
  
void moveTo(float x, float y, float z) {  
    moveX(x - currentPos.x);  
    moveY(y - currentPos.y);  
    moveZ(z - currentPos.z);  
  
    currentPos.x = x;  
    currentPos.y = y;  
    currentPos.z = z;  
}
```

27.10 End-Effector (Suction) Control

The suction end-effector is controlled via a digital output driving a pneumatic valve. Dedicated functions are used to activate and deactivate suction, ensuring clear separation between motion logic and actuator control.

This modular approach improves safety and maintainability while enabling reliable grasping and release during pick-and-place operations.

```
#define SUCTION_VALVE_PIN 23

void suctionON() {
    digitalWrite(SUCTION_VALVE_PIN, HIGH);
}

void suctionOFF() {
    digitalWrite(SUCTION_VALVE_PIN, LOW);
}
```

27.11 Pick-and-Place Motion Sequences

Standardized motion sequences are implemented for picking and placing parts:

- **Pick Sequence:**
Moves the robot to a safe height, approaches the target, activates suction, and retracts vertically.

```
void pickSequence(Position p) {
    moveTo(p.x, p.y, SAFE_Z);
    moveTo(p.x, p.y, p.z);
    suctionON();
    moveTo(p.x, p.y, SAFE_Z);
}
```

- **Place Sequence:**
Moves to the target position, releases the part, and retracts safely.

```
void placeSequence(Position p) {
    moveTo(p.x, p.y, SAFE_Z);
    moveTo(p.x, p.y, p.z);
    suctionOFF();
    moveTo(p.x, p.y, SAFE_Z);
}
```

- **Temporary Pick Sequence:**

Handles retrieval of stored parts from temporary locations during FIFO-based assembly.

```
void tempPick(Position tempPos) {  
    pickSequence(tempPos);  
}
```

These predefined sequences reduce code duplication and ensure consistent and collision-free motion.

27.12 Assembly Supervisor Logic

The supervisor function coordinates incoming parts, storage logic, and assembly operations. It implements FIFO handling for bases and lids, ensuring correct pairing and order preservation.

When both a base and a matching lid are available, the system automatically performs assembly by retrieving the required components and placing the assembled product at the designated location. Temporary storage is used when parts arrive out of sequence, ensuring robust handling of real-world operating conditions.

```
if (baseAvailable && lidAvailable) {  
    assembleProduct();  
}  
else if (lidArrivedFirst) {  
    tempLidBuffer[lidWriteIdx++] = lidPos;  
}
```

```
void assembleProduct() {  
    pickSequence(basePos);  
    placeSequence(assemblyPos);  
  
    pickSequence(lidPos);  
    placeSequence(assemblyPos);  
}
```

27.13 RTOS Architecture and Task Allocation

The system is structured using FreeRTOS to achieve deterministic and modular operation:

- **Homing Task:**
Executes once at startup to initialize all axes and signals system readiness.
- **Supervisor Task:**
Handles high-level sequencing, part arrival simulation, and assembly coordination.
- **Continuous Y-Axis PID Task:**
Runs independently with a fixed sampling time to ensure stable and accurate closed-loop control.

```
xTaskCreatePinnedToCore(HomingTask, "Homing", 2048, NULL, 4, NULL, 1);  
xTaskCreatePinnedToCore(SupervisorTask, "Supervisor", 4096, NULL, 3, NULL, 1);  
xTaskCreatePinnedToCore(Y_PID_Task, "Y_PID", 4096, NULL, 5, NULL, 1);
```

Semaphore synchronization is used to ensure tasks execute in the correct order, preventing race conditions and unsafe motion.

27.14 System Initialization and Hardware Configuration

During system startup, all GPIOs, encoders, motor drivers, limit switches, and actuators are initialized in a controlled manner.

Tasks are pinned to a specific core to avoid timing jitter and ensure predictable real-time behavior.

The main loop is intentionally left empty, as all system functionality is handled through RTOS tasks, reinforcing the activity-based architecture of the control system.

```
void setup() {  
    pinMode(SUCTION_VALVE_PIN, OUTPUT);  
    pinMode(X_LIMIT_PIN, INPUT_PULLUP);  
    pinMode(Y_LIMIT_PIN, INPUT_PULLUP);  
    pinMode(Z_LIMIT_PIN, INPUT_PULLUP);  
  
    motionSemaphore = xSemaphoreCreateBinary();  
}
```


27.15 Importance and Contribution to Overall System

This section of the code demonstrates:

- Safe multi-axis motion control
- Robust Homing and referencing
- RTOS-based task separation
- Closed-loop DC motor control
- Reliable pick-and-place automation
- FIFO-based assembly handling

Together, these elements ensure stable operation, fault tolerance, and compliance with the functional subsystem and RTOS requirements of the project evaluation criteria.

28 Conclusion

This project successfully demonstrated the design, implementation, and validation of a complete Cartesian robotic system capable of automated feeding, sorting, storage, and assembly operations. The mechanical design accurately reflected real-life constraints, ensuring proper fixation of sensors, actuators, and electronics while adhering to safety and manufacturability considerations.

From a control perspective, the system achieved reliable and deterministic operation through an RTOS-based architecture. Task separation, semaphore synchronization, and activity-based execution ensured safe coordination between motion control, sensing, communication, and assembly logic. Closed-loop PID control for the Y-axis provided stable motion with minimal steady-state error, no overshoot, and robust behavior during continuous operation.

The integration of ESP-NOW communication enabled seamless and low-latency data exchange between distributed controllers, supporting scalable and modular system expansion. Stress analysis of critical mechanical components confirmed high safety factors, minimal displacement, and Von Mises stress levels well below material limits, indicating no risk of structural failure under operating conditions.

Overall, the system met all functional and evaluation requirements, demonstrating strong integration between mechanical design, electrical implementation, and embedded software control. The project highlights the effectiveness of RTOS-based robotic systems in achieving reliable, flexible, and industrially relevant automation solutions.

29 References

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2018.
- [3] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton Univ. Press, 2010.
- [4] M. Barr and A. Massa, *Programming Embedded Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2006.
- [5] FreeRTOS, “FreeRTOS Kernel Documentation,” Real Time Engineers Ltd., 2024. [Online]. Available: <https://www.freertos.org>
- [6] Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems Inc., 2023. [Online]. Available: <https://www.espressif.com>
- [7] Espressif Systems, “ESP-NOW Wireless Communication Protocol,” Espressif Systems Inc., 2023. [Online]. Available: <https://docs.espressif.com>
- [8] R. Bishop, *The Mechatronics Handbook*, 2nd ed. Boca Raton, FL, USA: CRC Press, 2007.
- [9] S. Bennett, *A History of Control Engineering, 1930–1955*. London, U.K.: Peter Peregrinus Ltd., 1993.
- [10] Texas Instruments, “Understanding PID Control,” Application Report, 2020. [Online]. Available: <https://www.ti.com>
- [11] FESTO, *Pneumatics Basic Level – Training Manual*. Esslingen, Germany: FESTO Didactic, 2016.
- [12] NXP Semiconductors, “TCS3200 Color Sensor Datasheet,” 2019. [Online]. Available: <https://www.nxp.com>
- [13] Bosch Rexroth, *Industrial Motion Control Systems*. Stuttgart, Germany: Bosch Rexroth AG, 2018.
- [14] ISO 12100, *Safety of Machinery – General Principles for Design*, International Organization for Standardization, 2010.